



Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

LEARNER GUIDE

For

Generative AI for Image and Video Creation

TGS Ref No: TGS-2020505925

Conducted by

Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

Version 11.0

DOCUMENT VERSION CONTROL RECORD

Version Number	Effective Date of Release	Summary of Included Changes	Author
10.0	Legacy release (superseded)	Legacy Learner Guide for 'Image and Video Processing with OpenCV', built around Raspberry Pi provisioning and the six OpenCV topics. Retained as the content-coverage reference for this revision.	Dr. Alfred Ang
11.0	6 September 2026	Rebuilt for the registered title 'Generative AI for Image and Video Creation'. Full numbered procedures for twelve new self-contained labs, each with its own mock data, local media, prompt pack, expected outputs, evidence checklist and troubleshooting table. Added the provenance rules, the evidence-labelling discipline (MEASURED / MODELLED / PUBLISHED / ILLUSTRATIVE) and a page-cited source register. Legacy Raspberry Pi and Darknet build instructions removed. API currency verified on 6 September 2026, including the contrib-only status of cv2.TrackerCSRT_create, cv2.TrackerKCF_create and cv2.quality, and the deprecation of the imagen-4.0-* and veo-3.0-generate-001 model identifiers. Slide references regenerated against the 204-slide v11.0 deck.	Dr. Alfred Ang

TABLE OF CONTENTS

Introduction.....	4
How to Use This Guide.....	4
Course Learning Outcomes.....	4
Assessed Criteria in Full.....	5
Before You Start — Environment Setup.....	6
Provenance Rules That Apply to Every Artefact.....	7
Topic 01 — Generative AI Fundamentals for Image and Video Creation.....	8
Topic 02 — AI Image Generation, Editing and Visual Enhancement.....	13
Topic 03 — AI-Based Visual Features, Styles and Consistency.....	30
Topic 04 — Generative AI Video Creation, Animation and Editing.....	39
Topic 05 — Evaluating Cloud and Edge AI Creative Workflows.....	58
Preparing for the Assessment.....	63
Glossary.....	64
Source Register.....	66

Introduction

This Learner Guide accompanies the WSQ course Generative AI for Image and Video Creation (TGS-2020505925), conducted by Tertiary Infotech Academy Pte Ltd. It is version v11.0, released 6 September 2026.

The course is delivered over two days across five topics, and is assessed against the Skills Framework TSC Computer Vision Technology (ICT-DIT-4022-1.1) at Proficiency Level 4. The Written Assessment covers the thirteen knowledge statements K1 to K13; the Practical Performance covers the eight ability statements A1 to A8.

The slides carry the mechanisms, the measurements and the decision rules. This guide carries the FULL numbered procedure for every lab, so you can repeat any lab after the course without the trainer present. Both are open-book references in the assessment.

Note: Everything here runs on the base `opencv-python` wheel plus NumPy. No lab needs a paid service, and no lab downloads a model. Where a generative model can optionally be used, a local reference set is supplied so the deliverable can always be completed offline.

How to Use This Guide

- Read the topic section before its labs — the guide's topic sections state the mechanism the labs then measure.
- Work the labs in order. Later labs reuse measurements and files from earlier ones: Lab 12 times the pipeline you built across Labs 02 to 11.
- Every lab folder under labs/ carries the same README as the steps reproduced here, plus its data, its media, its prompt pack and a PDF of every Markdown file.
- Complete the evidence checklist at the end of each lab. Those items are what an assessor looks at.
- Where a figure appears, check its label: MEASURED, MODELLED, PUBLISHED or ILLUSTRATIVE. Never lift an ILLUSTRATIVE figure into a client document.

Course Learning Outcomes

- LO1: Understand basic vision systems concepts and applications
- LO2: Apply image processing
- LO3: Implement feature extraction
- LO4: Apply machine learning based computer vision methods
- LO5: Implement video analytics algorithms
- LO6: Evaluate edge vs cloud-based computer vision systems

ELO	Statement	Abilities	Knowledge	Delivery topic
ELO1	Understand basic vision systems concepts and applications	A1	K1, K2	Topic 01
ELO2	Apply image processing	A2	K3, K4	Topic 02
ELO3	Implement feature extraction	A3, A4	K5, K6, K7	Topic 03
ELO4	Apply machine learning based computer vision methods	A5	K8, K9	Topic 04
ELO5	Implement video analytics algorithms	A6	K10	Topic 04
ELO6	Evaluate edge vs cloud-based computer vision systems	A7, A8	K11, K12, K13	Topic 05

Assessed Criteria in Full

Knowledge — assessed in the Written Assessment (13 questions, 60 minutes)

Code	Knowledge statement	Question
K1	Vision system concepts	Question 1
K2	Business applications of vision systems	Question 2
K3	Methods to represent image and video data	Question 3
K4	Image and video processing, filtering and transformation methods	Question 4
K5	Feature extraction and representation techniques	Question 5
K6	Local feature descriptions, edge, colour, texture and motion	Question 6
K7	Global feature descriptions, statistical and geometrical methods	Question 7
K8	Deep learning concepts	Question 8
K9	Object segmentation, detection and recognition	Question 9
K10	Activity tracking, generative models, scene understanding and event discovery	Question 10
K11	Vision system architecture	Question 11
K12	Vision communication protocols	Question 12
K13	Real-world design constraints and solution options	Question 13

Abilities — assessed in the Practical Performance (5 tasks, 90 minutes)

Code	Ability statement	Task	Practised in
A1	Identify the needs of vision systems technology in industrial applications	Task 1	Lab 01
A2	Apply the principles of processing, filtering and analysis methods for video data	Task 2	Lab 02, Lab 03, Lab 04, Lab 05
A3	Analyse global feature descriptions	Task 2	Lab 07
A4	Design and implement feature extraction and representation methods	Task 3	Lab 06, Lab 07
A5	Design and apply machine-learning based methods for object detection, object tracking and activity recognition	Task 4	Lab 08
A6	Design and apply video analytics algorithms for high-level video analytics tasks	Task 4	Lab 09, Lab 10, Lab 11
A7	Design the architecture of applied vision systems	Task 5	Lab 12
A8	Design, develop and evaluate edge-based and cloud-based systems	Task 5	Lab 12

Note: Nothing is assessed that is not taught. Every question traces to a slide range and every task traces to a lab; the answer keys cite both.

Before You Start — Environment Setup

What you need

- A Windows or macOS laptop with Python 3.10 or later.
- opencv-python 4.10 or later, and NumPy. Nothing else is required.
- About 60 MB of disk for the labs/ folder, which includes every reference image and clip.
- Internet access is OPTIONAL and is used only for the optional generation steps in Labs 03, 04 and 09.

Install once, before the class if you can

```
python3 -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\Activate.ps1
pip install --upgrade pip
pip install "opencv-python>=4.10" numpy
python3 -c "import cv2, numpy; print(cv2.__version__, numpy.__version__)"
```

API currency — two things older tutorials get wrong

- `cv2.TrackerCSRT_create` and `cv2.TrackerKCF_create` are NOT in the base `opencv-python` wheel — they need `opencv-contrib-python`. Lab 10 therefore uses `cv2.TrackerMIL_create`, which is built in.
- The `cv2.quality` SSIM module is also contrib-only. The course ships its own NumPy SSIM implementation in `ssim.py` rather than adding a dependency for one function.
- `cv2.data.harcascades` points inside the installed package and holds the trained cascade XML files, and `cv2.HOGDescriptor_getDefaultPeopleDetector()` returns trained SVM coefficients — so Lab 08 runs two machine-learning detectors with no download at all.
- SURF remains non-free and is not in the standard wheel. Do not specify it; use ORB.

Conventions used in every lab

- Run commands from inside the lab folder, not from its parent.
- Each lab writes everything it produces into its own `out/` directory.
- `cv2.imread` returns `None` on failure and does NOT raise — always guard it.
- Array indexing is `[row, column] = [y, x]`; drawing functions take `(x, y)`.
- Where a step shows a `python3 -c` one-liner it is a standalone demonstration; the multi-line fragments assume one notebook or REPL session.

Provenance Rules That Apply to Every Artefact

1. A **SIMULATED** asset is a deterministic OpenCV rendering made for this course. It is never described as a photograph and never as generative-model output. Every such file carries a burned-in banner.
2. An **AI-GENERATED** reference is copied unchanged and travels with a `PROVENANCE.md` recording the tool, the date and the exact prompt.
3. A **REAL PRERECORDED** sequence records its source URL, retrieval date and SHA-256 so the copy can be verified.
4. An **ANIMATIC** is a deterministic render of storyboard frames, labelled on every frame. It exists to fix timing before generation spend and is never submitted as a generated film.
5. Anything you generate yourself is recorded with the model identifier, the prompt version and the watermark expectation. Current Gemini image outputs carry a SynthID watermark.
6. The label travels to every derivative. A crop of a simulated asset is still a simulated asset.

Note: Mislabelling provenance is an assessment failure and, in production, a governance breach. This is the one rule in the course with no exceptions.

Topic 01 — Generative AI Fundamentals for Image and Video Creation

LU1 · ELO1 · knowledge K1, K2 · abilities A1

Slides 24–54 · 75 minutes classroom facilitation · 0 minutes practical

Vision-system concepts and business applications · the analysis-to-synthesis pipeline · artificial neurons, activation functions and CNNs · VAE, GAN and diffusion architectures · what each model can and cannot guarantee

Key concepts

A vision system is a five-stage pipeline. Sense (sensor + optics) -> represent (pixel array, colour space) -> analyse (features, detection, tracking) -> decide (thresholds, policy) -> act or synthesise. Generative AI adds the synthesise stage; it does not remove the first four.

Classification and generation solve different conditional problems. A discriminative classifier learns $p(\text{label} | \text{image})$ or a decision boundary. A conditional generator models $p(\text{image} | \text{condition})$. These have different objectives: running a classifier backward does not generally generate an image.

Three architectures, three failure signatures. VAE: a regularised latent space; some variants lose fine detail. GAN: potentially sharp output, with risks of mode collapse and unstable training. Diffusion: potentially high fidelity, with cost from iterative sampling.

A neuron is a weighted sum plus a non-linearity. $y = f(\sum(w_i x_i) + b)$. Without the non-linearity f , any depth of stacked layers collapses to a single linear map. ReLU, sigmoid, tanh, softmax and GELU are the ones you will see in vision models.

A CNN learns the filters that OpenCV hand-codes. A 3x3 Sobel kernel is a hand-designed edge detector; a CNN's first-layer kernels can learn edge and colour-blob responses from training data. Related local filtering, learned instead of specified.

Business value comes from throughput, not novelty. Costed use cases: variant generation for catalogue and social, background replacement for product shots, format repurposing, storyboard previsualisation, synthetic data for detector training, and automated QC of the output.

Lab 01 — Vision-System Needs Analysis and Generative Pipeline Specification

Outcome: ELO1 · **Abilities:** A1 · **Knowledge:** K1, K2

Slide: 53 · **Folder:** [labs/lab-01-vision-needs-analysis/](#) · **In class:** facilitated within LU1 classroom facilitation (no separate practical minutes)

Scenario

NorthQuay Retail Group runs 41 stores and an online catalogue. Six departments have each asked for 'AI images'. You are the vision-systems analyst: score the six candidate use cases against measurable need criteria, reject the ones that do not need a vision system, and specify the five-stage pipeline for the two that survive.

What you will produce

A completed needs-analysis scoring matrix, a written rejection rationale, and a labelled five-stage pipeline specification for the two selected use cases.

Tools: Spreadsheet or text editor · Python 3 (optional scorer) · no paid service required

Files supplied with this lab

Path	What it is
reference/nq-storefront-reference.png	Synthetic reference frame representing a NorthQuay store-shelf camera view (deterministically rendered with OpenCV — labelled SIMULATED, not a photograph and not generative-model output)
data/candidate-use-cases.csv	Six candidate use cases with volume, latency tolerance, accuracy tolerance, privacy class, current manual cost and image-supply notes
data/need-criteria.json	The six weighted need criteria and the accept/reject threshold
brief/northquay-brief.md	The one-page client brief with constraints and the deliverable list
score_needs.py	Runnable script
PROMPTS.md / PROMPTS.pdf	The reusable prompt and specification pack

Step-by-step

1. Work inside this lab folder. Create out/ with your file manager. Python is optional: use the manual route below if unavailable. Create out/needs-scores.csv with headers case_id,name,volume,repeatibility,latency_class,accuracy_tolerance,privacy_risk,cost_gap,weighted_total,verdict. Enter six rows. For each row calculate the weighted total with SUMPRODUCT(score cells, weight cells) using weights from need-criteria.json; do not include non-score columns.

2. Before scoring, create out/eligibility.csv with columns case_id,vision_needed,cost_ok,turnaround_ok,privacy_ok,evidence. Record PASS, FAIL or UNKNOWN in every gate. Verify that visual inputs are necessary and compare projected per-asset cost, turnaround and permitted processing against the brief. UNKNOWN and FAIL mean HOLD/REJECT regardless of score; obtain missing evidence rather than inventing it. Annotate the final manual verdict with these gates. The Python scorer creates a preliminary feasibility baseline only; it never overrides these gates.

3. Open `brief/northquay-brief.md` and read the client constraints. Note the three hard constraints stated there: the S\$ per-asset ceiling, the maximum acceptable turnaround, and the customer-image privacy rule.

4. Open `data/candidate-use-cases.csv`. Confirm it has 6 data rows and all columns listed in the header (the scorer needs nine fields; `image_supply` may provide additional narrative evidence). Read every row before scoring anything.

```
python3 -c "import csv;r=list(csv.DictReader(open('data/candidate-use-cases.csv')));print(len(r), list(r[0]))"
```

5. Open `data/need-criteria.json`. The six criteria are volume, repeatability, latency_class, accuracy_tolerance, privacy_risk and cost_gap. Each is scored 0-5 and carries a weight; the accept threshold is stated in the same file.

6. Score criterion 1 — VOLUME. A vision system earns its build cost through throughput. Score 5 where the task repeats at least 500 times a month, 3 for 50 to under 500, 1 for under 50. Write your score in your matrix, not in the CSV.

7. Score criterion 2 — REPEATABILITY. Score 5 where the visual task is the same every time (same framing, same object class, same output format) and 1 where every instance is bespoke. Low repeatability can increase implementation cost and reduce whether a vision project pays back.

8. Score criterion 3 — LATENCY CLASS. Score 5 for batch/overnight (easy), 3 for interactive seconds, 1 for hard real-time under 100 ms. This score is inverted against difficulty on purpose: a high score means the latency requirement does not constrain your architecture.

9. Score criterion 4 — ACCURACY TOLERANCE. Score 5 where a human reviews every output before it ships (generous tolerance) and 1 where a wrong output reaches a customer or a safety decision unreviewed.

10. Score criterion 5 — PRIVACY RISK, inverted. Score 5 where no identifiable person or customer data is involved and 1 where customer faces or personal data are unavoidable. Cross-check against the `privacy_class` column in the CSV.

11. Score criterion 6 — COST GAP. Compute $(\text{current_manual_cost_sgd_per_month} - \text{estimated_automated_cost_sgd_per_month}) / \text{current_manual_cost_sgd_per_month}$. Score 5 at or above 0.6, 3 for 0.3 to below 0.6, 1 below 0.3.

12. Compute the weighted total for each case: $\text{total} = \sum(\text{score}_i * \text{weight}_i)$. Do this by hand for at least one case so you can explain the arithmetic, optionally run the provided scorer for a deterministic baseline; it does not read your manual scores. Reconcile every difference in writing.

```
python3 score_needs.py --csv data/candidate-use-cases.csv --criteria data/need-criteria.json --out out/baseline-scores.csv
```

13. Compare each weighted total against the accept threshold in [data/need-criteria.json](#). The supplied baseline is expected to select two cases. If your justified scores differ, compare with the baseline and document the evidence; do not change scores to force a count.
14. Write one sentence of rejection rationale for each rejected or held case. Name the criterion that failed and the number. 'Low volume' is not a rationale; 'volume 1/5 — 22 instances a month against a 500-instance classroom scoring boundary (not a measured payback point)' is.
15. For each of the selected eligible cases, specify the five-stage pipeline. Stage 1 SENSE: name the image source (existing catalogue asset, store camera, or a text prompt) and its available resolution and rate; for text input mark pixel fields not applicable and specify intended output dimensions.
16. Stage 2 REPRESENT: state the colour space and the working resolution, and where the asset is stored. Generate a candidate before its output-quality review. Stage 3 ANALYSE: name the specific measurement that will gate quality — for example an HSV histogram distance against the brand reference, or detector recall measured against an annotated validation set, never an unlabelled live frame.
17. Stage 4 DECIDE: state the numeric threshold and what happens on each side of it (auto-publish, human review queue, reject and regenerate). Mark each proposed threshold provisional until calibrated on representative labelled examples; policy rules may be categorical.
18. Stage 5 ACT: publish or route the reviewed artefact; on failure loop back to generation. State its format and aspect ratios, and who consumes it. Note explicitly whether the stage is analytical (measure and report) or generative (create a new asset).
19. Open [reference/nq-storefront-reference.png](#) and confirm the label banner reads SIMULATED. Use it as the stand-in shelf frame in your Stage 1 description and record in your write-up that it is a synthetic classroom asset, not a photograph and not a generative-model output.
20. Compare `out/baseline-scores.csv` if you ran Python; keep the manual gated decisions in `out/needs-scores.csv`. Save your completed matrix as [out/needs-analysis.md](#) (or `.csv`) together with the selected pipeline specifications, and complete the evidence checklist below.

Verify

Your `out/needs-scores.csv` shows all six cases with explicit eligibility gates and weighted scores against the accept threshold, every rejected or held case has a numbered rationale naming the failing criterion, and each surviving case has all five pipeline stages filled in with a named measurement and a numeric decision threshold.

Expected outputs

- `out/needs-scores.csv` — 6 rows, each with six criterion scores, a weighted total and an ACCEPT/REJECT verdict.
- Final ACCEPT verdicts require all four eligibility gates to PASS; explain any difference from the two-case baseline.
- `out/needs-analysis.md` — rejection or hold rationales, each naming a criterion and its score.
- Five-stage pipeline specifications for the selected eligible cases, each with a named Stage 3 measurement and a numeric Stage 4 threshold.

Evidence checklist

- Screenshot or paste of `out/needs-scores.csv` showing the weighted totals, plus `out/eligibility.csv`. A manually completed equivalent CSV is accepted.
- The accepted case IDs, their totals and all four eligibility gates; explain differences from the baseline.
- Rejection or hold rationales, one per rejected case, each citing a criterion score.
- Completed pipeline specifications for selected eligible cases covering all five stages.
- A one-line statement that the reference frame is a labelled synthetic asset.

Troubleshooting

Symptom	What to do
<code>score_needs.py</code> reports a <code>KeyError</code> on a column name	The CSV header must be unedited. Re-copy <code>data/candidate-use-cases.csv</code> from the lab folder; do not open and re-save it in a spreadsheet that renames columns.
Five or six cases pass the threshold	Compare the hand scores with the baseline; investigate the evidence, including the <code>image_supply</code> narrative if present. For this classroom rubric, a case with no repeatable image supply scores at most 2 on repeatability.
No case passes the threshold	Check that you inverted <code>privacy_risk</code> and <code>latency_class</code> as instructed — a high score means the constraint is EASY, not that the risk is high.
You cannot decide a Stage 3 measurement	Example measurements include: a histogram distance, an edge density, a keypoint match count, a detector precision/recall, or a PSNR/SSIM value. Other valid measures include latency, cost and temporal stability. Pick a measure matched to the risk and explain its limitations.

Note: The same content, plus the full prompt pack, is in `labs/lab-01-vision-needs-analysis/README.md` and its PDF. Sources used in this lab: S3, S5, S6, S14, S15.

Topic 02 — AI Image Generation, Editing and Visual Enhancement

LU2 · ELO2 · knowledge K3, K4 · abilities A2

Slides 55–94 · 75 minutes classroom facilitation · 60 minutes practical

How image and video data is represented · convolution, filtering and geometric transformation · prompt structure and controlled variance · masking, inpainting, outpainting and background swap · enhancement measured with PSNR and SSIM

Key concepts

An image is a typed 3-D array. OpenCV returns H x W x C uint8 in BGR order. shape, dtype and channel order are the three facts that cause most integration bugs; matplotlib expects RGB.

A video is frames plus a clock. `cap.get(cv2.CAP_PROP_FRAME_COUNT) / FPS` gives duration; a 1080p 30 fps RGB stream is $1920 \times 1080 \times 3 \times 30 = 186.6$ MB/s uncompressed. That number drives every cloud-vs-edge decision later.

HSV separates hue from saturation and brightness. For uint8 OpenCV HSV, H is 0-179 (degrees/2), S and V are 0-255. Hue can be more stable than raw RGB under brightness changes, but low saturation, shadows and colour casts still cause failures. Test the actual lighting range.

Kernel filtering combines neighbouring pixels. $\text{dst}(x,y) = \text{sum over the kernel of } K(i,j) \times \text{src}(x+i, y+j)$. A 5x5 box kernel of 1/25 blurs; Sobel approximates a first derivative; Laplacian a second derivative. Sharpening combines a scaled derivative response with the original. `cv2.filter2D` performs correlation; flip an asymmetric kernel for convolution.

Geometric transforms are matrix multiplies. Translation and rotation are 2x3 affine matrices used by `warpAffine`; perspective is a 3x3 homography used by `warpPerspective`. Affine preserves parallel lines; perspective preserves only straightness.

Generative editing is masked, conditioned denoising. These labs use white = edit and black = preserve for binary masks; provider interfaces may differ. Check their contract and verify unmasked output regions. Inpainting fills inside the frame; outpainting extends the canvas; background replacement targets the complement of the subject.

Lab 02 — Image and Video Data Representation

Outcome: ELO2 · **Abilities:** A2 · **Knowledge:** K3

Slide: 90 · **Folder:** [labs/lab-02-image-video-representation/](#) · **In class:** 15 minutes

Scenario

Before you can filter, edit or generate anything you have to know exactly what you are holding. Interrogate a still and a clip as data: array shape, dtype, channel order, colour space, frame rate and uncompressed bit-rate — the numbers that decide every later architecture choice. Classroom core (15 minutes): run the supplied measurement script, inspect the still/clip report and explain channel order and byte rate. The full implementation walkthrough and extra media trials are extension work.

What you will produce

A measurement report (`out/representation-report.json`) recording shape, dtype, channel means, HSV ranges, frame count, FPS, duration and the computed uncompressed byte rate for the supplied clip.

Tools: Python 3 · opencv-python · NumPy · offline after dependencies are installed; no paid service

Files supplied with this lab

Path	What it is
reference/nq-product-flatlay.png	Synthetic product flat-lay reference still, 1280x720 (deterministically rendered with OpenCV — SIMULATED classroom asset, not a photograph)
reference/nq-shelf-pan.mp4	Synthetic 6-second 640x360 25 fps shelf-pan clip (deterministically rendered animation — SIMULATED, not generative-model output)
data/expected-representation.json	Reference answers for the shape, dtype, FPS and frame-count checks so you can self-verify without the trainer
represent.py	Runnable script
PROMPTS.md / PROMPTS.pdf	The reusable prompt and specification pack

Step-by-step

1. Before class, install dependencies online or from a trainer-supplied wheelhouse. At class time use the prepared environment offline. On Windows activate with `.venv\Scripts\Activate.ps1`; the shell commands below are for macOS/Linux. Create and activate the lab environment, then confirm the OpenCV version. Everything in this lab runs on the base `opencv-python` wheel — no contrib modules and no downloaded weights.

```
python3 -m venv .venv && source .venv/bin/activate
pip install --upgrade pip
pip install "opencv-python>=4.10" numpy
python3 -c "import cv2; print(cv2.__version__)"
```

2. For the following Python fragments, open a notebook or persistent Python REPL, run `import cv2; img=cv2.imread("reference/nq-product-flatlay.png"); assert img is not None`. Shell commands labelled `python3 -c` are separate demonstrations and do not retain variables. Load the still with `cv2.imread` and print its `shape` and `dtype`. Record the three numbers: height, width and channel count.

```
python3 -c "import cv2;i=cv2.imread('reference/nq-product-flatlay.png');print(i.shape, i.dtype)"
```

3. Confirm the channel order is B, G, R — not R, G, B. Read the pixel at (row 10, col 10) in the top-left calibration patch, which is drawn pure blue. A BGR read returns a high first element; an RGB read would return a high third element.

```
python3 -c "import cv2;i=cv2.imread('reference/nq-product-flatlay.png');print('pixel (10,10) =', i[10,10])"
```

4. Split the image into its three channels with `cv2.split` and print the mean of each. Note which channel has the largest mean intensity — this is what a naive RGB threshold would be keying on.

```
b, g, r = cv2.split(img)
print('B mean %.1f G mean %.1f R mean %.1f' % (b.mean(), g.mean(), r.mean()))
```

5. Convert to grayscale with `cv2.COLOR_BGR2GRAY` and confirm the result is 2-D — the channel axis disappears. Grayscale is a weighted sum, not an average: OpenCV uses $0.299R + 0.587G + 0.114B$.

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
print(gray.shape, gray.dtype)
```

6. Verify that weighting yourself on the calibration patches. Compute the luma of a pure green patch by hand ($0.587 * 255 = 149.7$) and compare it against the grayscale value OpenCV produced. Locate a pure green pixel programmatically and report its coordinates; fail if the required patch is absent.

```
import numpy as np
ys,xs=np.where(np.all(img == (0,255,0),axis=2)); assert len(xs), 'missing green patch'
y,x=int(ys[0]),int(xs[0]); print(y,x,int(gray[y,x]))
```

7. Convert to HSV with `cv2.COLOR_BGR2HSV`. Print the min and max of each channel. Confirm the OpenCV 8-bit convention: H is 0-179 (degrees halved so it fits a byte), S and V are 0-255. Confusing this encoding with degree values causes incorrect thresholds.

```
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
for n, c in zip('HSV', cv2.split(hsv)):
    print(n, int(c.min()), int(c.max()))
```

8. Read the hue of the red, green and blue calibration patches. Red sits near $H=0$ (and wraps to 179), green near $H=60$, blue near $H=120$. Write the three measured values down; you will threshold on them in Lab 06. Find an exact pure-colour pixel for each patch and report coordinates; do not guess coordinates.

```
for name,bgr in [('red',(0,0,255)),('green',(0,255,0)),('blue',(255,0,0))]:
    ys,xs=np.where(np.all(img==bgr,axis=2)); assert len(xs), name+' patch missing'
    y,x=int(ys[0]),int(xs[0]); print(name,y,x,hsv[y,x].tolist())
```

9. Open the clip with `cv2.VideoCapture` and read its properties: frame width, frame height, FPS and frame count. Do not trust a filename or a container label for these.

```
cap = cv2.VideoCapture('reference/nq-shelf-pan.mp4')
```

```
w = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
h = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
fps = cap.get(cv2.CAP_PROP_FPS)
n = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
print(w, h, fps, n, 'duration %.2fs' % (n / fps))
```

10. Count the frames you can actually read in a loop and compare that count with `CAP_PROP_FRAME_COUNT`. The property is backend-dependent and may be derived from metadata and can disagree with the decodable frame count — record the loop result and investigate any mismatch in work that matters.

```
read = 0
while True:
    ok, frame = cap.read()
    if not ok:
        break
    read += 1
print('reported', n, 'decoded', read)
```

11. Compute the uncompressed byte rate: width x height x channels x bytes-per-channel x fps (uint8 BGR uses 3 bytes/pixel). For the supplied 640x360 25 fps clip that is $640 \times 360 \times 3 \times 25 = 17,280,000$ B/s = 17.3 MB/s. Then compute the same figure for 1920x1080 at 30 fps and write both numbers down — Topic 5 uses them to justify sending events instead of frames.

```
for (W, H, F) in [(640, 360, 25), (1920, 1080, 30)]:
    print(W, H, F, '->', W * H * 3 * F / 1e6, 'MB/s uncompressed')
```

12. Measure the actual file size on disk and divide by the duration to get the encoded byte-rate (multiply by 8 for bits/s). Compare it with the uncompressed rate you just computed and state the compression ratio.

```
python3 -c "import os;print(os.path.getsize('reference/nq-shelf-pan.mp4'),
'bytes')"
```

13. Run the full report script. It performs every measurement above and writes `out/representation-report.json`.

```
python3 represent.py --image reference/nq-product-flatlay.png --video
reference/nq-shelf-pan.mp4 --out out/representation-report.json
```

14. Compare your report against `data/expected-representation.json`. The shape, dtype, FPS, decoded frame count must match the fixed reference; compare rates/FPS with tolerance 0.01 and means with tolerance 0.1; the channel means may differ in the last decimal place across OpenCV builds.

```
python3 -c "import json;a=json.load(open('out/representation-
report.json'));b=json.load(open('data/expected-
representation.json'));keys=['image_shape','image_dtype','video_frames_decoded'
];bad=[k for k in keys if k not in b or a.get(k)!=b[k]];bad += [k for k in
['video_fps','uncompressed_bytes_per_second'] if k not in b or abs(a[k]-
b[k])>0.01]; print(bad or 'ALL MATCH')"
```

15. Save `out/representation-report.json` and complete the evidence checklist.

Verify

`out/representation-report.json` exists and its `image_shape`, `image_dtype`, `video_fps`, `video_frames_decoded` and `uncompressed_bytes_per_second` fields match `data/expected-representation.json`, and you can state the OpenCV hue range from memory.

Expected outputs

- `image_shape` = [720, 1280, 3] and `image_dtype` = "uint8".
- Pixel (10,10) reads as a high-B, low-G, low-R triple, confirming BGR order.
- Measured HSV extrema lie within allowed uint8 ranges H 0-179, S/V 0-255; actual extrema need not reach those limits.
- `video_fps` = 25.0, 150 frames decoded, duration 6.00 s.
- `uncompressed_bytes_per_second` = 17280000 for the supplied clip.

Evidence checklist

- `out/representation-report.json` attached or pasted.
- The measured hue values of the red, green and blue calibration patches.
- The two uncompressed byte-rate figures (640x360@25 and 1920x1080@30).
- The compression ratio of the supplied clip (encoded size vs uncompressed).
- A one-line statement of why `CAP_PROP_FRAME_COUNT` is not authoritative.

Troubleshooting

Symptom	What to do
<code>cv2.imread</code> returns None	The path is wrong or the file is unreadable — <code>imread</code> does not raise. Always guard with <code>if img is None: raise SystemExit('cannot read <path>')</code> and run from the lab folder, not from its parent.
Colours look wrong when displayed with matplotlib	matplotlib expects RGB; OpenCV gives BGR. Show with <code>plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))</code> .
Hue values above 179 or a hue of 255 for red	Check for HSV_FULL, wrong-channel interpretation, or a float image. Swapping RGB/BGR changes hue identity but does not exceed the uint8 H bound. Convert from the raw <code>cv2.imread</code> result and keep dtype uint8.
<code>cap.get(cv2.CAP_PROP_FPS)</code> returns 0	The backend could not read the container metadata. Inspect trusted timestamps/container metadata or re-copy the known reference asset. Never infer playback FPS by timing decode throughput. Do not time N frames as a substitute. A damaged MP4 may lack its moov atom.
Decoded frame count is one less than reported	For this supplied fixed asset, 150 decodable frames are required. Report both numbers and re-copy/recheck the input or decoder if they differ.

Note: The same content, plus the full prompt pack, is in `labs/lab-02-image-video-representation/README.md` and its PDF. Sources used in this lab: S23, S14, S18.

Lab 03 — Prompt Experiment Matrix for Controlled Image Generation

Outcome: ELO2 · **Abilities:** A2 · **Knowledge:** K3, K4

Slide: 91 · **Folder:** [labs/lab-03-prompt-experiment-matrix/](#) · **In class:** 15 minutes

Scenario

A marketing team complains that 'the AI keeps changing the product'. Treat prompting as a designed experiment: compare progressively specified prompt bundles, generate or simulate a cell of the matrix, and score output variance numerically instead of arguing about it. You will end with a versioned JSON prompt that a colleague can reproduce. Classroom core (15 minutes): specify one constrained prompt and evaluate the supplied comparison set; generate one live result when account access permits. Repeated live trials and the full implementation walkthrough are extensions, not assumed complete in 15 minutes.

What you will produce

A completed factor-level table and four-run comparison, a versioned [prompt-v3.json](#) structured prompt, and [out/variance-report.csv](#) scoring the measured spread of each cell.

***Tools:** Text editor · Python 3 · opencv-python · NumPy · OPTIONAL free-tier image model. The lab completes fully offline using the supplied simulated variant sets.

Files supplied with this lab

Path	What it is
reference/variants/cella_01.png .. cellD_04.png	16 supplied variant images — four cells of four variants each, deterministically rendered with OpenCV to reproduce the characteristic spread of an under-specified, partially specified, subject-locked and fully specified prompt. Every file carries a SIMULATED banner: these are classroom stand-ins for model output, NOT generative-model output.
data/prompt-factors.json	The four prompt factors and their levels, in the structure used by the scorer
data/campaign-brief.md	The NorthQuay 'Harbour Line' campaign brief the prompts must satisfy
data/prompt-v1.json	The under-specified starting prompt
data/prompt-v2.json	The partially specified prompt with subject lock
prompt_variance.py	Runnable script
PROMPTS.md / PROMPTS.pdf	The reusable prompt and specification pack

Step-by-step

1. Start inside this lab folder using the prepared Python/OpenCV/NumPy environment. Create [out/](#) with your file manager. Record source filenames, model/settings if used and selected path in [out/provenance.md](#). These simulations teach measurement only; their trend does not demonstrate a real model effect.

2. Read `data/campaign-brief.md`. Extract the four things the brief actually constrains: the SUBJECT (the product and who holds it), the CONTEXT (setting and time of day), the STYLE (visual register), and the TECHNICAL modifiers (lens, lighting, aspect ratio). These four are your experiment factors.
 3. Open `data/prompt-v1.json` — the under-specified baseline. Note that it names only the subject. Predict, before measuring, which of the four factors will vary most between runs. Write your prediction down.
 4. Build the experiment matrix. Rows are the four factors; columns are three levels: ABSENT, NAMED, LOCKED. 'Named' means the factor appears as a phrase; 'locked' means it appears as an explicit constrained value (for example `"lens": "35mm"` rather than 'wide shot'). Record the matrix in `out/experiment-matrix.md`.
 5. Choose the four cells you will actually run: A = all factors absent except subject; B = subject named + context named; C = subject LOCKED + context named + style named; D = all four LOCKED. A full factorial design would have $3^4=81$ combinations; these four bundles illustrate a comparison within the lab slot, not an isolated causal effect.
 6. Write the four prompts. Record every change between bundles and explicitly note that several factors change together; causal attribution to one factor is not possible.
 7. OPTIONAL — if you have free-tier access to an image model, generate four variants per cell with a recorded sampling policy (seed only if supported) and save them as `out/gen/cellX_0N.png`. If you do not, skip to the next step and use the supplied simulated set. Before live generation create `out/prompt-A.json` through `out/prompt-D.json` from the prompt pack; for prose cells use a JSON object with a prompt key. Either path completes the lab; record which one you used.
- ```
Current documented Gemini image models (verified 6 Sep 2026):
gemini-3.1-flash-image, gemini-3.1-flash-lite-image, gemini-3-pro-image
(gemini-2.5-flash-image is the legacy model)
All generated images carry a SynthID watermark.
The imagen-4.0-* IDs used by older tutorials are deprecated.
See PROMPTS.md for the exact request bodies.
```
8. For the offline route only, load the supplied variant sets from `reference/variants/`. Confirm there are 16 files in four cells of four, and that each carries the SIMULATED banner.

```
ls reference/variants | wc -l
```

9. Score within-cell variance on SUBJECT. For each cell, compute the mean pairwise Bhattacharyya distance between the HSV histograms of the subject region (the central 60% crop). Low distance means similar HS colour distributions in that crop, not proven subject identity or position.

```
python3 prompt_variance.py --variants reference/variants --out out/variance-report.csv
For your generated files instead use:
python3 prompt_variance.py --variants out/gen --out out/variance-report.csv
```

10. Exclude the banner from interpretation: the following metrics use the whole frame and its background. They are coarse brightness/layout proxies, not validated subject or style scores. Score within-cell variance on COMPOSITION. The scorer also reports the standard deviation of the subject centroid across each cell, in pixels. This is a limited proxy and cannot establish a prompt that keeps the product but moves it around the frame.
11. Score within-cell variance on STYLE. The scorer reports the standard deviation of edge density and of mean saturation across the cell — a proxy for 'is the visual register holding'.
12. Read `out/variance-report.csv`. For the designed simulated set inspect its intended trend: cell A has the highest histogram distance and centroid spread, and cell D the lowest. If your own generated set does not show that trend, record the result honestly; sampling uncertainty and model behaviour may explain it. Review the full change log without changing observations to fit expectations.
13. Compare the measured result against your step-2 prediction. Write one sentence assessing your prediction; state whether your prediction was supported; do not invent an error if it was correct.
14. Freeze your chosen prompt (based on brief fit, not variance alone) as `out/prompt-v3.json`. Use one key per factor so a single value can be changed without rewriting a sentence — that is the whole argument for structured prompting over prose.
15. Add the three enterprise fields to the frozen prompt: `prompt_version`, `brief_reference` and `negative_constraints`. A prompt with a version number is easier to audit alongside its hash and provenance.
16. Record the provenance expectation for anything you actually generated: which model, which prompt version, and the fact that current Gemini image outputs carry a SynthID watermark. If you used the simulated set, record that instead — never label a simulated asset as model output.
17. Save `out/experiment-matrix.md`, `out/variance-report.csv` and `out/prompt-v3.json`, then complete the evidence checklist.

## Verify

`out/variance-report.csv` shows the mean pairwise histogram distance and centroid standard deviation reported for every cell A to D; real-model results need not be monotonic, and `out/prompt-v3.json` parses as valid JSON with one key per factor plus `prompt_version`, `brief_reference` and `negative_constraints`.

## Expected outputs

- `out/experiment-matrix.md` — a 4x3 factor-by-level matrix with the four run cells marked.

- `out/variance-report.csv` — one row per cell with `hist_distance_mean`, `centroid_std_px`, `edge_density_std` and `saturation_std`.
- Report the observed trend and limitations; the simulated trend is designed, not evidence of prompting efficacy.
- `out/prompt-v3.json` — valid JSON, one key per factor, versioned.
- `out/provenance.md` with exact input filenames, settings, dimensions and limitations. A written statement of which generation path was used (free-tier model or supplied simulated set).

### Evidence checklist

- The 4x3 experiment matrix with the four run cells identified.
- `out/variance-report.csv` pasted, showing the A-to-D trend.
- Your step-2 prediction and the one-sentence correction after measuring.
- `out/prompt-v3.json` including `prompt_version` and `negative_constraints`.
- An explicit provenance line: model + prompt version + watermark expectation, OR a statement that the supplied simulated variant set was used.

### Troubleshooting

| Symptom                                                             | What to do                                                                                                                                                                                                                                      |
|---------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| The variance trend is flat or reversed on your own generated images | This can be a valid stochastic result. Record it, inspect all bundle changes and model settings; an adjective such as 'cinematic' silently locks style in a cell that was supposed to leave it absent, but this is not the only possible cause. |
| <code>prompt_variance.py</code> reports 'need 4 variants per cell'  | The one flat variants folder needs four files for each of cellA, cellB, cellC and cellD with the <code>cell&lt;X&gt;_&lt;NN&gt;.png</code> naming. Re-copy <code>reference/variants/</code> from the lab folder.                                |
| Bhattacharyya distance is 0.0 for every pair                        | Different spatial arrangements or brightness can share the same HS histogram. Check for duplicates, and state that this feature does not verify identity or spatial structure.                                                                  |
| A free-tier request is refused or rate-limited                      | Expected, and not a blocker. The lab is designed to complete on the supplied simulated set; record that path and move on. Never let a paid or rate-limited service gate a classroom deliverable.                                                |
| You are tempted to submit a simulated variant as model output       | Do not. Mislabelling provenance is an assessment failure and, in production, a governance breach. The SIMULATED banner is burned into the supplied images deliberately.                                                                         |

**Note:** The same content, plus the full prompt pack, is in `labs/lab-03-prompt-experiment-matrix/README.md` and its PDF. Sources used in this lab: S7, S12, S17, S20, S21.

## Lab 04 — Masking, Inpainting, Outpainting and Background Swap

**Outcome:** ELO2 · **Abilities:** A2 · **Knowledge:** K3, K4

**Slide:** 92 · **Folder:** [labs/lab-04-masking-inpainting-outpainting/](#) · **In class:** 15 minutes

### Scenario

A product shot has a stray price tag in frame, the wrong background, and it is square when the campaign needs a 16:9 hero. Masks help specify all three edits, but output quality also depends on the repair method. Build the masks yourself in OpenCV, run the deterministic classical repair to see the mechanism, measure what changed inside and outside the mask, then write the generative edit specification that a model would execute. Classroom core (15 minutes): run the three bundled mask jobs, inspect preservation measurements and write the edit/preserve specification. Building the implementation from scratch and additional provider trials are extensions.

### What you will produce

Three masks ([out/mask\\_tag.png](#), [out/mask\\_bg.png](#), [out/mask\\_outpaint.png](#)), the classical inpaint results, [out/mask-audit.csv](#) proving the unmasked region was preserved, and [out/edit-spec.json](#) specifying the generative equivalent.

**\*Tools:** Python 3 · opencv-python · NumPy · offline with preinstalled dependencies; no paid service

### Files supplied with this lab

| Path                                                     | What it is                                                                                                                                             |
|----------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">reference/nq-jacket-square.png</a>           | Synthetic 1024x1024 product shot with a subject, a plain backdrop and a stray price-tag rectangle (deterministically rendered with OpenCV — SIMULATED) |
| <a href="#">reference/nq-backdrop-harbour.png</a>        | Synthetic 1820x1024 replacement backdrop (SIMULATED)                                                                                                   |
| <a href="#">data/edit-jobs.json</a>                      | Three edit jobs with their target regions, prompts and acceptance thresholds                                                                           |
| <a href="#">data/mask-conventions.md</a>                 | The binary-mask convention reference: white = editable, black = preserved                                                                              |
| <a href="#">mask_edit.py</a>                             | Runnable script                                                                                                                                        |
| <a href="#">PROMPTS.md</a> / <a href="#">PROMPTS.pdf</a> | The reusable prompt and specification pack                                                                                                             |

### Step-by-step

1. Start in this lab folder with prepared OpenCV/NumPy dependencies. Create out/. For Python fragments use one notebook or REPL: `import cv2, numpy as np; from pathlib import Path; Path('out').mkdir(exist_ok=True); from mask_edit import read, background_mask; img=read('reference/nq-jacket-square.png')`. The full script performs all jobs; fragment demonstrations share this context.

2. Read `data/mask-conventions.md`. The convention used in this lab and by `cv2.inpaint` is: a single-channel 8-bit image where WHITE (255) marks the editable pixels and BLACK (0) marks preserved pixels. Check provider-specific semantics before using an API, and verify output preservation. Getting this inverted can cause masked-edit failure.

3. Load `reference/nq-jacket-square.png` and confirm it is 1024x1024x3 uint8. Locate the price tag visually and note its approximate bounding box.

```
python3 -c "import cv2;i=cv2.imread('reference/nq-jacket-square.png');print(i.shape)"
```

4. Build mask 1 — the price tag — by COLOUR rather than by hand-typed coordinates. The tag is drawn in a saturated yellow; threshold in HSV so the mask survives if the product moves. Save it as `out/mask_tag.png`.

```
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
mask = cv2.inRange(hsv, (20, 120, 120), (35, 255, 255)) # H 20-35 = yellow
cv2.imwrite('out/mask_tag.png', mask)
```

5. Clean the mask with morphology: `cv2.MORPH_CLOSE` with a 5x5 kernel fills pinholes, then a single `cv2.dilate` grows the mask a few pixels past the object edge. That dilation is what lets the fill blend — a mask cut exactly on the boundary leaves a halo.

```
k = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, k)
mask = cv2.dilate(mask, k, iterations=2)
cv2.imwrite('out/mask_tag.png', mask)
```

6. Report the mask coverage as a percentage of the frame. A tag mask should be a low single-digit percentage; if it is 30% your hue band is catching the product too.

```
print('coverage %.2f%%' % (100.0 * (mask > 0).mean()))
```

7. Run classical inpainting twice — `cv2.INPAINT_TELEA` (fast marching) and `cv2.INPAINT_NS` (Navier-Stokes). Both propagate surrounding texture inward; neither invents new content. That difference is exactly what separates classical repair from generative inpainting.

```
telea = cv2.inpaint(img, mask, 3, cv2.INPAINT_TELEA)
ns = cv2.inpaint(img, mask, 3, cv2.INPAINT_NS)
```

8. Vary the `inpaintRadius` argument across 1, 3 and 9 and look at the three results. Larger radius pulls texture from further away: smoother on flat backdrops, smeared across a textured one. Record your chosen radius and review its artefacts.

```
for radius in (1,3,9):
 cv2.imwrite(f'out/telea_radius_{radius}.png', cv2.inpaint(img, mask,
radius, cv2.INPAINT_TELEA))
```

9. Audit the edit. Compute the mean absolute difference between the original and the result INSIDE the mask and OUTSIDE the mask. Outside must be effectively zero — that is the numeric proof that the edit was contained.

```
python3 mask_edit.py --image reference/nq-jacket-square.png --jobs data/edit-jobs.json --out out
```

10. Build mask 2 — the background — as the complement of the subject. Segment the subject with Otsu on the saturation channel, take the largest connected component, fill it, then invert. Save as `out/mask_bg.png`.

```
s = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)[: , : , 1]
_, fg = cv2.threshold(s, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
bg, fg = background_mask(img) # largest component, filled holes
cv2.imwrite('out/mask_bg.png', bg)
```

11. Composite the supplied replacement backdrop through mask 2 to see the mechanical background swap. Note where it fails: the hard matte edge and any colour spill the original lighting left on the subject. A generative background swap may improve the boundary by re-rendering it but may introduce new errors; alpha matting and colour correction can also improve composites.

```
bg_img = cv2.resize(cv2.imread('reference/nq-backdrop-harbour.png'),
 (img.shape[1], img.shape[0]))
swapped = np.where(bg[:, :, None] > 0, bg_img, img)
```

12. Build mask 3 — the outpatient canvas. Pad the 1024x1024 square onto a 1820x1024 canvas (approximately 16:9, 1.77734:1) with the original centred, then create a mask that is BLACK over the original pixels and WHITE over the new empty side panels. This is the classroom canvas contract; each outpainting API may require different inputs.

```
canvas = np.zeros((1024, 1820, 3), np.uint8)
x0 = (1820 - 1024) // 2
canvas[:, x0:x0 + 1024] = img
om = np.full((1024, 1820), 255, np.uint8)
om[:, x0:x0 + 1024] = 0
cv2.imwrite('out/mask_outpaint.png', om)
```

13. Confirm the outpaint mask geometry numerically: the white region must be exactly 2 x 398 x 1024 pixels and the black region exactly 1024 x 1024. If the numbers are off by one, your centring arithmetic is wrong and the seam will show.

```
print('white', int((om > 0).sum()), 'expected', 2 * 398 * 1024)
```

14. Open `data/edit-jobs.json` and read the three acceptance thresholds. Job 1 (tag removal) requires outside-mask mean absolute difference < 0.5 intensity levels on the 0-255 scale; job 2 (background) requires the subject's HSV histogram to stay within a Bhattacharyya distance of 0.15 of the original; job 3 (outpaint) requires the central 1024 columns to be bit-identical to the source.

15. Check every job against its threshold in `out/mask-audit.csv` and mark each PASS or FAIL. The CSV reports geometry/pixel checks, not semantic quality. Inspect the segmented silhouette against the actual jacket; an incorrect matte can still pass preservation metrics. Job 3 is only a blank canvas, not a generated outpaint. A failure requires checking masks, source alignment, encoding and algorithm/model behaviour; it does not identify one cause.

16. Write `out/edit-spec.json` — the generative equivalent of what you just did classically. For each job record: the mask file, the mask semantics (which colour is

editable), the edit mode (inpaint-removal, background-swap, outpaint), the edit prompt, the preserve list, and the acceptance metric with its threshold.

17. Add the mask-free alternative to the spec for job 1 only, as a second entry: the same instruction expressed as a text-only edit with no mask supplied. Note in the spec the trade-off — mask-free may be faster to author; either method needs review before commercial use because a generative mask is not a universal pixel-preservation guarantee.

18. Save the three masks, the inpaint results, `out/mask-audit.csv` and `out/edit-spec.json`, then complete the evidence checklist.

## Verify

`out/mask-audit.csv` shows job 1 with an outside-mask mean absolute difference below 0.5 intensity levels on the 0-255 scale and a clearly non-zero inside-mask difference, the outpaint white-pixel count equals 815,104, and `out/edit-spec.json` parses with a mask file, an edit mode, a preserve list and a numeric acceptance threshold for all three jobs.

## Expected outputs

- `out/mask_tag.png` — single-channel, coverage in low single-digit percent.
- `out/mask_bg.png` — subject black, background white.
- `out/mask_outpaint.png` — 1024x1820, exactly 815,104 white pixels.
- `out/inpaint_tela.png` and `out/inpaint_ns.png` with the tag removed.
- `out/mask-audit.csv` — inside/outside mean absolute difference per job with PASS/FAIL.
- `out/edit-spec.json` — three jobs plus one mask-free alternative entry.

## Evidence checklist

- The three mask images, with the coverage percentage of each.
- The inside-mask and outside-mask mean absolute differences for job 1.
- The `inpaintRadius` you chose (1, 3 or 9) and one sentence of justification.
- The outpaint white-pixel count matching 815,104.
- `out/edit-spec.json` showing the mask semantics stated explicitly for every job.
- One sentence on where the mechanical background composite visibly fails and why a generative swap might help or fail.

## Troubleshooting

| Symptom                             | What to do                                                                                                                                                                                                                                       |
|-------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| The inpainted region is a grey blob | Possible causes include a wrong mask, insufficient surrounding texture or an unsuitable radius. <code>cv2.inpaint</code> repairs where the mask is non-zero. Print <code>mask.mean()</code> : a tag mask should be a small number, not near 255. |

| Symptom                                                          | What to do                                                                                                                                                                                                             |
|------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>cv2.inpaint</code> raises an assertion about the mask type | The mask must be single-channel 8-bit (CV_8UC1). If you built it from a colour operation, convert with <code>cv2.cvtColor(mask, cv2.COLOR_BGR2GRAY)</code> first.                                                      |
| Outside-mask difference is not zero                              | <code>cv2.inpaint</code> only writes inside the mask, so a non-zero outside difference means you compared against the wrong source image, or you resized somewhere. Re-read the original from disk for the comparison. |
| The background mask captures part of the subject                 | Otsu on saturation splits at a single global threshold. Take only the largest connected component of the foreground and fill its holes with <code>cv2.floodFill</code> before inverting.                               |
| The outpaint seam is visible after compositing                   | That is the expected classical result and is the teaching point: this simple hard composite does not continue texture across the seam. Record it rather than trying to hide it.                                        |

**Note:** The same content, plus the full prompt pack, is in [labs/lab-04-masking-inpainting-outpainting/README.md](#) and its PDF. Sources used in this lab: S8, S17, S20, S23.

## Lab 05 — Filtering, Enhancement and Restoration Measured with PSNR and SSIM

**Outcome:** ELO2 · **Abilities:** A2 · **Knowledge:** K4

**Slide:** 93 · **Folder:** [labs/lab-05-filtering-enhancement-metrics/](#) · **In class:** 15 minutes

### Scenario

'Looks better' is not a deliverable. Take a clean reference frame, damage it three ways, repair it with five filters, and rank the repairs with PSNR and SSIM computed from the formulas — then examine whether the two metrics disagree and explain which one you would trust for a customer-facing asset. Classroom core (15 minutes): run the supplied enhancement comparison, inspect two contrasting results and explain why metrics alone cannot select the best creative output. The full implementation walkthrough and further parameter trials are extensions.

### What you will produce

`out/enhancement-matrix.csv` scoring 5 filters against 3 degradations with PSNR and SSIM, plus a one-paragraph recommendation naming the winning filter per degradation and whether their rankings differ.

**\*Tools:** Python 3 · opencv-python · NumPy · offline with preinstalled dependencies; no paid service

### Files supplied with this lab

| Path                        | What it is                                                                                                                                                                                                    |
|-----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| reference/nq-hero-clean.png | Synthetic 800x600 clean reference frame with flat regions, hard edges, fine texture and a smooth gradient — the four content types that separate filters (deterministically rendered with OpenCV — SIMULATED) |
| data/degradations.json      | The three degradations with their exact parameters and random seed                                                                                                                                            |
| data/filter-bank.json       | The five candidate filters with their kernel sizes and parameters                                                                                                                                             |
| data/expected-ranking.csv   | Reference winning filter per degradation, for self-verification                                                                                                                                               |
| enhance.py                  | Runnable script                                                                                                                                                                                               |
| ssim.py                     | Runnable script                                                                                                                                                                                               |
| PROMPTS.md / PROMPTS.pdf    | The reusable prompt and specification pack                                                                                                                                                                    |

## Step-by-step

1. Use the prepared offline environment and work inside this lab folder. For fragments use one notebook/REPL with `import cv2, numpy as np; img=cv2.imread('reference/nq-hero-clean.png');` `assert img is not None`. The full script saves each degraded/filtered image; select one pair as `a=img` and `b=cv2.imread('out/fixed_<degradation>_<filter>.png')` before PSNR comparison.

2. Load `reference/nq-hero-clean.png`. This is the ground truth: every metric in this lab is computed against it, so nothing is filtered before it is measured.

```
python3 -c "import cv2;i=cv2.imread('reference/nq-hero-clean.png');print(i.shape)"
```

3. Read `data/degradations.json`. The three degradations are: GAUSSIAN noise (sigma 18), SALT-AND-PEPPER impulse noise (4% of pixels), and DEFOCUS blur (Gaussian, sigma 2.5). The random seed is fixed; allow small implementation-dependent float differences and report measured results.

4. Apply degradation 1 — additive Gaussian noise. Note that you must clip to [0,255] and cast back to uint8; casting out-of-range sums back to uint8 without clipping wraps around and produces speckle that is not the noise you intended.

```
rng = np.random.default_rng(20260906)
noise = rng.normal(0, 18, img.shape)
gauss = np.clip(img.astype(np.float64) + noise, 0, 255).astype(np.uint8)
```

5. Apply degradation 2 — salt and pepper. Choose 4% of pixel positions at random and set half to 0 and half to 255. This is impulse noise: a few pixels are catastrophically wrong while the rest are untouched. The supplied function samples unique positions (rounded down to whole pixels).

```
from enhance import degrade_salt_pepper
impulse=degrade_salt_pepper(img,0.04,20260906)
```

6. Apply degradation 3 — defocus blur with `cv2.GaussianBlur(img, (0,0), 2.5)`. Passing (0,0) for ksize makes OpenCV derive the kernel size from sigma, which is what you want when sigma controls the simulated blur strength.

```
defocus=cv2.GaussianBlur(img,(0,0),2.5)
```

7. Now the five filters. Filter 1 — BOX blur, `cv2.blur(img, (5,5))`. This is a 5x5 kernel of 1/25, the simplest low-pass. Write out the kernel by hand and confirm it sums to 1; a kernel that does not sum to 1 changes the image brightness.

8. Filter 2 — GAUSSIAN blur, `cv2.GaussianBlur(img, (5,5), 0)`. Weights fall off with distance from the centre, so its smoothing differs from the box filter at the same support. With `sigma=0` OpenCV derives `sigma` from `ksize`.

9. Filter 3 — MEDIAN, `cv2.medianBlur(img, 5)`. This is a rank filter, not a convolution: it replaces each pixel with the median of its neighbourhood. Predict now which degradation it will dominate, and why a median is more resistant to isolated outliers than a mean is not.

10. Filter 4 — BILATERAL, `cv2.bilateralFilter(img, 9, 75, 75)`. It weights neighbours by spatial distance AND by intensity difference, so it smooths within a region but not across an edge. It can be costly — time it and record the cost.

11. Filter 5 — UNSHARP MASK. Build it yourself: blur the image, then `cv2.addWeighted(img, 1.5, blurred, -0.5, 0)`. This is sharpening expressed as 'original plus the high-frequency residual', and it can amplify noise; any unsuitable filter can make a noisy image worse.

```
blur = cv2.GaussianBlur(img, (0, 0), 3)
sharp = cv2.addWeighted(img, 1.5, blur, -0.5, 0)
```

12. Compute PSNR from the formula before you use the library call.  $MSE = \text{mean}((a-b)^2)$  over all pixels and channels;  $PSNR = 10 * \log_{10}(255^2 / MSE)$ . Verify your hand-rolled value against `cv2.PSNR` to within 0.01 dB.

```
mse = np.mean((a.astype(np.float64) - b.astype(np.float64)) ** 2)
psnr = 10 * np.log10(255.0 ** 2 / mse)
print(psnr, cv2.PSNR(a, b))
```

13. Compute SSIM. The base `opencv-python` wheel does NOT ship `cv2.quality`, so the lab provides a NumPy implementation in `ssim.py`. Read it: SSIM combines luminance, contrast and structure terms as  $((2*\mu_x*\mu_y + C1)(2*\sigma_{xy} + C2)) / ((\mu_x^2 + \mu_y^2 + C1)(\sigma_x^2 + \sigma_y^2 + C2))$  with  $C1=(0.01*255)^2$  and  $C2=(0.03*255)^2$ , computed over a sliding Gaussian window.

14. Run the full 3x5 matrix. Every cell reports PSNR and SSIM of the filtered result against the clean reference, plus the filter's wall-clock time.

```
python3 enhance.py --clean reference/nq-hero-clean.png --degradations
data/degradations.json --filters data/filter-bank.json --out out/enhancement-
matrix.csv
```

15. Read the matrix. Test the common hypothesis that MEDIAN performs well on salt-and-pepper; the ranking depends on the image and parameters. BILATERAL may preserve edges in Gaussian noise, but these filters cannot guarantee recovering detail lost to blur; deblurring is an inverse problem rather than just smoothing one.

16. Find the disagreement. Check whether PSNR and SSIM rank two filters differently; report agreement if none differ. For a disagreement explain: PSNR reflects mean squared error and may favour aggressive smoothing; SSIM is a windowed structural comparison that may penalise lost texture that the smoothing cost you.

17. Decide which metric you would gate a customer-facing asset on, and write one paragraph justifying it. Note the wider point for generated media: distributional metrics such as FID and perceptual metrics such as LPIPS exist for exactly this reason — a single per-pixel number does not describe perceived quality.

18. Compare your winners against `data/expected-ranking.csv`, save `out/enhancement-matrix.csv` and your recommendation paragraph, and complete the evidence checklist.

```
python3 -c "import csv;print(open('data/expected-ranking.csv').read())"
```

### Verify

`out/enhancement-matrix.csv` has 18 rows (15 filtered comparisons plus 3 baselines) each with a PSNR and an SSIM, your hand-computed PSNR agrees with `cv2.PSNR` to 0.01 dB, MEDIAN has a reported rank on salt-and-pepper, and you have checked whether PSNR and SSIM disagree.

### Expected outputs

- `out/enhancement-matrix.csv` — 18 rows: degradation, filter, psnr\_db, ssim, ms.
- Baseline row per degradation (no filter) so every improvement is measured against it.
- Observed MEDIAN rank on salt-and-pepper, with evidence.
- Observed BILATERAL rank on Gaussian noise.
- Measured improvement or worsening relative to defocus baseline; no recovery guarantee.
- Measured UNSHARP change relative to each noisy baseline.

### Evidence checklist

- `out/enhancement-matrix.csv` pasted in full.
- Your hand-computed PSNR for one cell, next to the `cv2.PSNR` value.
- The 5x5 box kernel written out, with its sum.
- Rankings by both metrics, including any disagreement or a statement of agreement.
- One paragraph recommending a metric to gate customer-facing assets, with reasoning.
- The bilateral filter's wall-clock time compared with the box filter's.

### Troubleshooting

| Symptom     | What to do                                                                                                                                  |
|-------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| PSNR is inf | MSE is zero: inputs are identical, whether intentionally or by an input mistake. Check you are passing the filtered result, not the source. |

| Symptom                                                 | What to do                                                                                                                                                                                                                       |
|---------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Adding noise makes the image look like static confetti  | Adding floats promotes the array; casting out-of-range results back to uint8 can wrap values. Cast to float64, add, <code>np.clip(..., 0, 255)</code> , then cast back to uint8.                                                 |
| <code>cv2.medianBlur</code> raises an error             | ksize must be odd and greater than 1, and for ksize > 5 the input must be uint8. Use 3, 5 or 7.                                                                                                                                  |
| <code>cv2.quality</code> is not available               | Correct — the SSIM module lives in <code>opencv-contrib-python</code> , not in the base wheel. This lab therefore ships its own NumPy SSIM in <code>ssim.py</code> ; use that. Do not add a contrib dependency for one function. |
| SSIM values look far too high for a badly damaged image | Check inputs, alignment, dynamic range and pooling; a high score alone does not establish the cause. Convert both images to grayscale first, or average the per-channel SSIM maps — <code>ssim.py</code> does the latter.        |
| The bilateral filter takes several seconds              | That is the finding, not a fault. Record the time; it is the argument for running restoration at the edge only when the latency budget allows it.                                                                                |

**Note:** The same content, plus the full prompt pack, is in [labs/lab-05-filtering-enhancement-metrics/README.md](#) and its PDF. Sources used in this lab: S17, S23, S14.

## Topic 03 — AI-Based Visual Features, Styles and Consistency

**LU3 · ELO3 · knowledge K5, K6, K7 · abilities A3, A4**

Slides 95–120 · 75 minutes classroom facilitation · 60 minutes practical

Feature extraction and representation · local descriptors — edge, colour, texture, corner and motion · global descriptors — statistical and geometrical · measuring style and composition consistency across a campaign

### Key concepts

**A feature is a repeatable, discriminative measurement.** Good features are repeatable under rotation, scale and illumination change, and different enough between classes to separate them. Everything downstream inherits the quality of this choice.

**Local versus global is a decision about scope.** Local descriptors answer 'what is at this point' (Canny edge, ORB keypoint, optical-flow vector). Global descriptors answer 'what is this whole image like' (colour histogram, Hu moments, edge density). Consistency work needs global.

**Corner detectors differ in what they are invariant to.** Harris: rotation-invariant, not scale-invariant. Shi-Tomasi: Harris with a min-eigenvalue score. FAST: segment test, very fast, no descriptor. SIFT/ORB: scale- and rotation-invariant with a matchable descriptor.

**Canny is a four-step algorithm, not a filter.** Gaussian smooth -> Sobel gradient magnitude and direction -> non-maximum suppression -> hysteresis thresholding with two thresholds. The 1:2 to 1:3 low:high ratio is the standard starting point.

**Global statistics are how you audit a brand look.** Normalised HSV histograms compared with Bhattacharyya distance, mean saturation, edge density and Hu moments give a numeric style fingerprint you can threshold and put in a review gate.

**Faithfulness needs a metric, not an opinion.** Text-image alignment via CLIP-style cosine similarity, object/attribute recall via a detector, and masked-region consistency for edits — each catches a different way a generated asset can be wrong.

## Lab 06 — Local Features: Colour Segmentation, Canny Edges and Corner

### Keypoints

**Outcome:** ELO3 · **Abilities:** A4 · **Knowledge:** K5, K6

**Slide:** 118 · **Folder:** [labs/lab-06-local-features-edges-keypoints/](#) · **In class:** 30 minutes

### Scenario

The campaign QC bot has to answer three questions about every delivered asset: is the brand colour present and in the right place, is the product silhouette crisp, and can this frame be matched to the approved master. Those are colour, edge and keypoint questions. Implement all three and record the transformations each tolerates and its failure cases.

### What you will produce

`out/features-report.json` with the HSV segmentation coverage, the Canny threshold sweep, response/count measurements for five detectors, and an ORB match count between the asset and its rotated copy.

\*Tools:\* Python 3 · opencv-python · NumPy · no network, no paid service

### Files supplied with this lab

| Path                                        | What it is                                                                                                                                                                                                            |
|---------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>reference/nq-asset-master.png</code>  | Synthetic 900x700 campaign asset containing the slate-blue brand block, a hard-edged product silhouette, a fine-texture panel and a chequered calibration corner (deterministically rendered with OpenCV — SIMULATED) |
| <code>reference/nq-asset-variant.png</code> | The same asset rotated 20 degrees and scaled to 0.8, for the matching step (deterministically rendered — SIMULATED)                                                                                                   |
| <code>data/brand-colour.json</code>         | The brand hue band, the required coverage range and the required location quadrant                                                                                                                                    |
| <code>data/expected-features.json</code>    | Reference keypoint counts and match counts for self-verification (tolerances included, since detector counts vary slightly across OpenCV builds)                                                                      |
| <code>features.py</code>                    | Runnable script                                                                                                                                                                                                       |

| Path                     | What it is                                 |
|--------------------------|--------------------------------------------|
| PROMPTS.md / PROMPTS.pdf | The reusable prompt and specification pack |

## Step-by-step

1. Load `reference/nq-asset-master.png` and convert it to HSV. Remember the OpenCV 8-bit convention from Lab 02: H is 0-179, S and V are 0-255.

```
img = cv2.imread('reference/nq-asset-master.png')
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
```

2. Read `data/brand-colour.json`. The slate-blue brand hue band is given as an H range with S and V floors. Build the mask with `cv2.inRange` and report coverage as a percentage of the frame.

```
mask = cv2.inRange(hsv, tuple(spec['lo']), tuple(spec['hi']))
print('brand coverage %.2f%%' % (100.0 * (mask > 0).mean()))
```

3. Test exposure robustness on this fixture, not a universal invariance claim. Multiply the V channel by 0.6 to simulate an under-exposed delivery, re-run the same `inRange`, and compare coverage. Then do the same experiment thresholding the BLUE channel of the BGR image instead. Record both coverage changes. Hue separation helps here, but V floors, clipping, noise and white-balance shifts can still break segmentation.

```
dark = hsv.copy(); dark[:, :, 2] = (dark[:, :, 2] * 0.6).astype('uint8')
print('HSV coverage under -40%% V: %.2f%%' % (100.0 * (cv2.inRange(dark, lo, hi) > 0).mean()))
```

4. Handle the red wrap-around case explicitly, even though this asset is blue. Red spans H near 179 and H near 0, so it needs two `inRange` calls combined with `cv2.bitwise_or`. Write the two-band code and keep it — every HSV pipeline meets red eventually.

```
m1 = cv2.inRange(hsv, (0, 120, 80), (8, 255, 255))
m2 = cv2.inRange(hsv, (172, 120, 80), (179, 255, 255))
red = cv2.bitwise_or(m1, m2)
```

5. Check the LOCATION requirement from `data/brand-colour.json`: the brand block must sit in the stated quadrant. Compute the mask centroid with `cv2.moments` and test which quadrant it falls in. Coverage alone does not prove correct placement.

6. Move to edges. Smooth grayscale with `cv2.GaussianBlur(gray, (5,5), 1.0)` first, then run `cv2.Canny` at (50,150), (100,200) and (150,300) and record the edge-pixel density of each. Canny takes two thresholds because it uses hysteresis: pixels above the high threshold seed strong edges; weaker pixels above the low threshold that connect to strong edges continue one.

7. Name the four Canny stages in your report — Gaussian smoothing, Sobel gradient magnitude and direction, non-maximum suppression, hysteresis thresholding. Canny is an algorithm, not a single convolution; you cannot tune it sensibly without knowing which stage each parameter touches.

8. Sweep the low:high ratio at a fixed high threshold of 200: try low = 200, 100, 66 (ratios 1:1, 1:2, 1:3). Record the edge density for each and pick the ratio that keeps the product silhouette continuous without lighting up the texture panel.

```
python3 features.py --image reference/nq-asset-master.png --brand data/brand-colour.json --out out/features-report.json
```

9. Now corners (the Harris count is response pixels, not distinct corners). Run `cv2.cornerHarris(gray32, blockSize=2, ksize=3, k=0.04)`, dilate the response, and count the pixels above 1% of the maximum response. Harris measures intensity variation in ALL directions — that is what distinguishes a corner from an edge.

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
dst = cv2.cornerHarris(np.float32(gray), 2, 3, 0.04)
dst = cv2.dilate(dst, None)
print('harris response pixels', int((dst > 0.01 * dst.max()).sum()))
```

10. Run `cv2.goodFeaturesToTrack` (Shi-Tomasi) with `maxCorners=200`, `qualityLevel=0.01`, `minDistance=10`. Shi-Tomasi uses the minimum eigenvalue of the structure tensor with an explicit spacing constraint, which returns a separated point set rather than a response map.

11. Rotate the image 30 degrees with `cv2.getRotationMatrix2D` + `cv2.warpAffine`, re-run Harris, and compare the count. Then SCALE the image to 0.5 and re-run. Record both. Harris is rotation-covariant in theory, but interpolation, cropping and thresholding change counts. Fixed-window Harris is not scale-invariant; count equality alone does not measure repeatability of corresponding corners.

12. Run `cv2.FastFeatureDetector_create()`. FAST tests a ring of 16 pixels around a candidate and accepts it if a contiguous arc is all brighter or all darker than the centre by a threshold. It is a detector only — it produces no descriptor, so it cannot match on its own.

13. Run `cv2.ORB_create(nfeatures=500)` with `detectAndCompute`. ORB gives you both keypoints and a 32-byte binary descriptor, and it is free of patent restrictions. Print `descriptors.shape` and confirm the second dimension is 32.

14. Run `cv2.SIFT_create()` with `detectAndCompute` and print `descriptors.shape`. The second dimension is 128 floats. Compare the descriptor sizes: 32 bytes versus 512 bytes per keypoint is a 16x storage and bandwidth difference at the same keypoint count — record it, because Topic 5 turns it into a cost number.

15. Match the master against `reference/nq-asset-variant.png` (rotated 20 degrees, scaled 0.8) using ORB plus `cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)`. Hamming distance is the correct metric for a binary descriptor; L2 is not.

```
bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)
matches = sorted(bf.match(des1, des2), key=lambda m: m.distance)
print('matches', len(matches), 'best distance', matches[0].distance)
```

16. Apply a median-distance heuristic: keep matches at or below  $0.75 \times$  the median distance. This is NOT Lowe's nearest/second-nearest ratio test. Report the surviving count and inspect correspondence geometry; neither filter proves identity.

17. Compare every count against `data/expected-features.json`, which carries tolerances because keypoint counts shift slightly between OpenCV builds. Save `out/features-report.json` and complete the evidence checklist.

## Verify

`out/features-report.json` contains brand coverage before and after the exposure change (explain the observed difference and the V-floor limitation), edge densities for three Canny settings, keypoint counts for Harris, Shi-Tomasi, FAST, ORB and SIFT, the Harris rotation and scale comparison, and a filtered ORB match count above zero between the master and the rotated variant.

## Expected outputs

- Brand-colour coverage within the range stated in `data/brand-colour.json`.
- Measure both coverage changes; the synthetic V scaling preserves H and S by construction, but pixels crossing the V floor can still leave the mask.
- Edge density is nondecreasing as thresholds fall on the same smoothed image; inspect which ratio keeps the silhouette continuous.
- Report Harris response-pixel counts under rotation and scaling without treating them as matched corner counts.
- ORB descriptor width 32; SIFT descriptor width 128.
- The supplied OpenCV 4.13 fixture produces 15 filtered ORB matches (79 raw); allow build variation, and inspect geometry. The illustrative gate minimum is 10.

## Evidence checklist

- `out/features-report.json` attached.
- The two coverage figures for the exposure test (HSV vs raw blue channel).
- The centroid quadrant of the brand mask and whether it met the placement rule.
- The four Canny stages named, and the low:high ratio you selected with a reason.
- Harris counts at 0 degrees, 30 degrees and 0.5 scale.
- ORB and SIFT descriptor widths, and the 16x storage comparison.
- The raw and median-heuristic-filtered ORB match counts and an inspected correspondence visual.

## Troubleshooting

| Symptom                                           | What to do                                                                                                                |
|---------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| <code>cv2.cornerHarris</code> raises an assertion | It requires a single-channel float32 input. Convert with <code>np.float32(cv2.cvtColor(img, cv2.COLOR_BGR2GRAY))</code> . |

| Symptom                                                                 | What to do                                                                                                                                                                                                                                                       |
|-------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| SIFT is not available                                                   | SIFT has been in the main OpenCV distribution since 4.4 — if <code>cv2.SIFT_create</code> is missing you are on an old wheel. <code>pip install --upgrade opencv-python</code> . Do NOT substitute SURF: it remains non-free and is not in the standard package. |
| <code>bf.match</code> raises an error about descriptor type             | You mixed a binary and a float descriptor, or passed <code>NORM_HAMMING</code> to SIFT. Use <code>NORM_HAMMING</code> for ORB/BRIEF/BRISK and <code>NORM_L2</code> for SIFT.                                                                                     |
| <code>detectAndCompute</code> returns <code>None</code> for descriptors | No keypoints were found — usually a blank or uniformly flat crop. Check the image loaded, and lower the detector threshold.                                                                                                                                      |
| Brand coverage is 0%                                                    | Your hue band is in degrees rather than OpenCV's halved scale. A 210-degree slate blue is <code>H=105</code> in OpenCV, not 210.                                                                                                                                 |
| Match count collapses to near zero after filtering                      | <code>crossCheck=True</code> plus a 0.75x-median filter is deliberately strict. Report the number; the count alone does not show which matches are geometrically correct. Zero-distance matches survive the inclusive cutoff even when the median is zero.       |

**Note:** The same content, plus the full prompt pack, is in [labs/lab-06-local-features-edges-keypoints/README.md](#) and its PDF. Sources used in this lab: S17, S23, S14.

## Lab 07 — Global Descriptors, Template Matching and Brand-Consistency

### Scoring

**Outcome:** ELO3 · **Abilities:** A3, A4 · **Knowledge:** K7

**Slide:** 119 · **Folder:** [labs/lab-07-global-descriptors-consistency/](#) · **In class:** 30 minutes

### Scenario

Twelve assets have come back from three different creators for the same campaign. Three contain deliberate fixture deviations. Build a global-descriptor fingerprint — colour histogram, saturation, edge density and Hu moments — score every asset against the approved master, and use template matching to verify the logo is present and correctly placed.

### What you will produce

`out/consistency-scores.csv` ranking all 12 assets against the master with four global descriptors and a combined score, plus `out/logo-placement.csv` from template matching, and a reviewed list of fixture deviations with the descriptor or logo gate that flagged each, including any misses.

**\*Tools:** Python 3 · `opencv-python` · NumPy · no network, no paid service

## Files supplied with this lab

| Path                                          | What it is                                                                                                                                                                                           |
|-----------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| reference/master/nq-master.png                | The approved campaign master, 900x700 (deterministically rendered — SIMULATED)                                                                                                                       |
| reference/assets/asset_01.png .. asset_12.png | Twelve delivered assets, nine on-brand and three deliberately off-brand in different ways — one recoloured, one over-sharpened, one with the logo misplaced (deterministically rendered — SIMULATED) |
| reference/nq-logo-template.png                | The 120x60 logo template used for matching (deterministically rendered — SIMULATED)                                                                                                                  |
| data/consistency-thresholds.json              | Per-descriptor pass thresholds and the combined-score weights                                                                                                                                        |
| data/asset-manifest.csv                       | The twelve assets with their creator, delivery date and declared variant                                                                                                                             |
| consistency.py                                | Runnable script                                                                                                                                                                                      |
| PROMPTS.md / PROMPTS.pdf                      | The reusable prompt and specification pack                                                                                                                                                           |

## Step-by-step

1. Read `data/asset-manifest.csv`. Twelve assets, three creators. Do NOT look at the images first. Rank numerical deviations without inferring quality from creator identity; then inspect the flagged assets and record false positives or missed defects.

2. Fingerprint the master. Descriptor 1 is the normalised 2-D hue-saturation histogram, 30 hue bins x 32 saturation bins, computed with `cv2.calcHist` and normalised with `cv2.normalize(..., 0, 1, cv2.NORM_MINMAX)`. Normalisation is what makes the comparison size-independent.

```
hsv = cv2.cvtColor(master, cv2.COLOR_BGR2HSV)
hist = cv2.calcHist([hsv], [0, 1], None, [30, 32], [0, 180, 0, 256])
cv2.normalize(hist, hist, 0, 1, cv2.NORM_MINMAX)
```

3. Compare a histogram with itself using all four `cv2.compareHist` methods and record the self-comparison values: CORREL gives 1.0, CHISQR gives 0.0, INTERSECT gives the histogram sum, BHATTACHARYYA gives 0.0. Knowing which direction 'good' points in for each method prevents an inverted gate.

```
for name, m in [('CORREL', cv2.HISTCMP_CORREL), ('CHISQR', cv2.HISTCMP_CHISQR),
 ('INTERSECT', cv2.HISTCMP_INTERSECT),
 ('BHATTACHARYYA', cv2.HISTCMP_BHATTACHARYYA)]:
 print(name, cv2.compareHist(hist, hist, m))
```

4. Use BHATTACHARYYA for the gate: it is bounded in [0,1], 0 means identical, and it behaves well on sparse histograms. Score all twelve assets against the master and record the distance.

5. Descriptor 2 — mean HSV saturation. This single number can flag saturation changes, but cannot detect hue-only recolouring. Compute it for the master and for all twelve, and record the absolute difference.

6. Descriptor 3 — Canny edge density, using the thresholds you settled on in Lab 06. This can flag texture/edge changes; it cannot uniquely identify sharpening or blur. Record the ratio `asset_density / master_density` rather than the raw difference, so the threshold is dimensionless; compare equal image sizes because edge density remains resolution-sensitive.

7. Descriptor 4 — Hu moments. These are seven values computed from normalised central moments with ideal translation, scale and rotation invariance; rasterisation and segmentation affect them. Otsu gives a global brightness mask, not a verified subject silhouette. Inspect that mask before interpreting shape similarity, and compare with `cv2.matchShapes`.

```
_, binimg = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
hu = cv2.HuMoments(cv2.moments(binimg)).flatten()
print(cv2.matchShapes(master_bin, binimg, cv2.CONTOURS_MATCH_I1, 0.0))
```

8. Take the log of the Hu moments before you compare them by hand: raw Hu values span many orders of magnitude, so `-sign(h) * log10(|h|)` is the standard readable form. Record the seven log values for the master.

9. Combine the four descriptors into a single consistency score using the weights in `data/consistency-thresholds.json`. Normalise each descriptor to [0,1] first — you cannot weight a Bhattacharyya distance against a saturation difference in raw units.

```
python3 consistency.py --master reference/master/nq-master.png --assets
reference/assets --thresholds data/consistency-thresholds.json --out
out/consistency-scores.csv
```

10. Read `out/consistency-scores.csv` sorted by combined score. Three fixture assets were deliberately modified, but need not all cross this numerical threshold. Join descriptor and logo-gate rejects by asset name, inspect them, and explain misses.

11. Inspect the ranked images and compare numeric flags with visible defects. Record false positives and false negatives without blaming a creator from metadata alone.

12. Now template matching. Load `reference/nq-logo-template.png` and run `cv2.matchTemplate(asset, template, cv2.TM_CCORR_NORMED)` for each asset. The result is a response map, not a box: every position gets a similarity score.

```
res = cv2.matchTemplate(asset_gray, tpl_gray, cv2.TM_CCORR_NORMED)
minv, maxv, minl, maxl = cv2.minMaxLoc(res)
print('best score %.3f at %s' % (maxv, maxl))
```

13. Note the two methods where LOW is best. For `TM_SQDIFF` and `TM_SQDIFF_NORMED` you take `minLoc`; for the other four you take `maxLoc`. Getting this backwards puts your detection in the opposite corner of the frame.

14. Apply the score threshold from `data/consistency-thresholds.json`. Report, per asset: the peak score, the peak location, whether it exceeds the threshold, and whether the top-left location falls inside the required placement box. This box constrains the logo origin, not full-box containment.

15. State template matching's limits explicitly in your write-up: it is not scale- or rotation-invariant, and it degrades under illumination change unless you use a normalised method. That is exactly why Lab 06's ORB matching exists alongside it.

16. Save `out/consistency-scores.csv` and `out/logo-placement.csv`, name the three fixture deviations with the descriptor or logo gate that caught each (or explain a miss), and complete the evidence checklist.

## Verify

`out/consistency-scores.csv` ranks all 12 assets with threshold-based decisions, each flag traceable to its weighted descriptor contribution, and `out/logo-placement.csv` shows one asset whose logo peak score passes but whose peak location falls outside the required placement box.

## Expected outputs

- `out/consistency-scores.csv` — 12 rows: asset, hist\_bhattacharyya, saturation\_delta, edge\_density\_ratio, shape\_match, combined\_score, verdict.
- Join both CSV gate verdicts by asset and compare flagged assets against all three fixture defects; report misses honestly.
- Inspect whether histogram distance flags recolouring; saturation delta only changes if saturation changes.
- Inspect whether edge density flags the modified texture; do not infer a sharpening cause from density alone.
- `out/logo-placement.csv` — 12 rows: peak score, peak (x,y), score\_pass, placement\_pass.
- The misplaced-logo asset passing on score and failing on placement.

## Evidence checklist

- `out/consistency-scores.csv` sorted by combined score.
- The four self-comparison values for the histogram methods, showing which direction is 'good'.
- The seven log-Hu moments of the master.
- All three named outliers, each with the mechanism that caught it: two by a named descriptor, one by the logo placement gate.
- `out/logo-placement.csv` showing the score-pass / placement-fail asset.
- One sentence stating the two template-matching methods where minLoc is used.
- One sentence stating template matching's invariance limits.

## Troubleshooting

| Symptom                          | What to do                                                                                                                                                       |
|----------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Every asset scores as an outlier | You compared un-normalised histograms, or mixed a normalised master against un-normalised assets. Normalise every histogram with the same call before comparing. |

| Symptom                                                            | What to do                                                                                                                                                                                                                                              |
|--------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>cv2.compareHist</code> raises a type error                   | Both histograms must be float32 of the same shape. <code>cv2.calcHist</code> returns float32; do not cast to float64 in between.                                                                                                                        |
| Hu moments are all zero                                            | The binary mask is empty. Check the Otsu threshold actually split the image — on a low-contrast asset, threshold the saturation channel instead of the grayscale.                                                                                       |
| <code>matchShapes</code> returns a huge number                     | One of the two inputs has no contour. Pass a binary image with a real foreground, inspect the segmentation and note that <code>CONTOURS_MATCH_I1</code> is not bounded to [0,1].                                                                        |
| Template match peaks in the wrong corner                           | You used <code>minLoc</code> with a <code>CCOEFF/CCORR</code> method, or <code>maxLoc</code> with a <code>SQDIFF</code> method. <code>SQDIFF</code> and <code>SQDIFF_NORMED</code> use <code>minLoc</code> ; everything else uses <code>maxLoc</code> . |
| Template match score is low on an asset you know contains the logo | The logo has been scaled or rotated in that delivery. Template matching cannot absorb either — record it as a limitation finding and match with ORB instead.                                                                                            |

**Note:** The same content, plus the full prompt pack, is in [labs/lab-07-global-descriptors-consistency/README.md](#) and its PDF. Sources used in this lab: S9, S17, S23.

## Topic 04 — Generative AI Video Creation, Animation and Editing

**LU4 + LU5 · ELO4 + ELO5 · knowledge K8, K9, K10 · abilities A5, A6**

Slides 121–161 · 150 minutes classroom facilitation · 180 minutes practical

Deep learning concepts · object segmentation, detection and recognition · detector evaluation with IoU and precision/recall · storyboard-to-prompt workflow · temporal coherence, tracking, activity and event discovery · captions and export profiles

### Key concepts

**Classification, detection and segmentation differ by output shape.** Classification returns one label per image; detection returns a list of boxes with labels and scores; segmentation returns a label per pixel. The evaluation metric changes with the output shape.

**Viola-Jones is a real machine-learning detector.** Haar-like rectangle features + integral image + AdaBoost feature selection + a cascade of stages that rejects background windows early. It ships inside OpenCV, so it runs with no downloaded weights.

**IoU is the contract between a prediction and the truth.**  $\text{IoU} = \text{area}(\text{intersection}) / \text{area}(\text{union})$ . At a stated confidence threshold, match detections one-to-one to ground truth with the correct class and an agreed IoU threshold, for example  $\geq 0.5$ .

Duplicates/unmatched predictions are false positives; unmatched ground-truth objects are false negatives.

**Video adds temporal coherence to spatial coherence.** Each frame must look right AND be consistent with its neighbours. Flicker, identity drift and warping backgrounds are temporal failures that a per-frame quality score cannot see.

**3-D U-Nets and Diffusion Transformers buy that coherence.** Extending the 2-D U-Net to 3-D adds the time axis so the model learns spatiotemporal features; treating video as a sequence of spatiotemporal patches lets a transformer model long-range dependencies across many seconds.

**Motion analysis is how you verify continuity numerically.** Background subtraction isolates the moving foreground; mean shift and CAMShift climb the back-projected histogram to track it; Farneback optical flow gives a dense displacement field. Centroid jitter and scale drift then become numbers.

## Lab 08 — Object Detection and Quantitative Evaluation

**Outcome:** ELO4 · **Abilities:** A5 · **Knowledge:** K8, K9

**Slide:** 157 · **Folder:** [labs/lab-08-object-detection-evaluation/](#) · **In class:** 60 minutes

### Scenario

Two machine-learning detectors that ship inside OpenCV — the Viola-Jones Haar cascade and the HOG + linear-SVM people detector — are run against a labelled ground truth. You will compute IoU, build the precision/recall curve by sweeping the detector's own detector-specific operating parameter (not a calibrated probability), and report the operating point you would actually deploy.

### What you will produce

`out/detections.json` with every predicted box, `out/evaluation.csv` with TP/FP/FN, precision, recall and F1 at three IoU thresholds, and a stated operating point with the parameter values that produce it.

**\*Tools:** Python 3 · opencv-python · NumPy — both detectors ship inside the base wheel, so no model weights are downloaded and no paid service is used

### Files supplied with this lab

| Path                                                                    | What it is                                                                                                                                                                                                                                                                               |
|-------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>reference/frames/frame_00.png</code> .. <code>frame_11.png</code> | Twelve 640x480 synthetic scene frames containing schematic face-like and person-like targets at known pixel positions (deterministically rendered with OpenCV — SIMULATED classroom assets, not photographs). Because the ground truth is generated with the frames, every box is exact. |

| Path                                                   | What it is                                                                                                                                                                |
|--------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| reference/positive-control/retail-adults-reference.png | AI-generated realistic retail scene with two fictional adults; a separate face-detector positive-control illustration, not a photograph or representative validation set. |
| data/positive-control-truth.json                       | Approximate independent manual visual boxes for two faces and two people in the realistic synthetic reference; review annotation boundaries before performance claims.    |
| data/ground-truth.json                                 | Exact bounding boxes for every target in every frame, with a class label                                                                                                  |
| data/eval-config.json                                  | The IoU thresholds to report, the parameter sweep ranges, and the operating point selection rule                                                                          |
| detect_eval.py                                         | Runnable script                                                                                                                                                           |
| PROMPTS.md / PROMPTS.pdf                               | The reusable prompt and specification pack                                                                                                                                |

## Step-by-step

1. Use a prepared Python/OpenCV/NumPy environment offline. Work in this lab folder; create out/. For fragments use one Python notebook/REPL with import cv2; img=cv2.imread('reference/frames/frame\_00.png'); assert img is not None; gray=cv2.cvtColor(img,cv2.COLOR\_BGR2GRAY). Model weights ship with OpenCV; schematic targets may yield no detections. Zero denominators use a reported 0.0 convention; no qualifying point means DO NOT DEPLOY.

2. Confirm the two detectors are available without any download.

cv2.data.harcascades points inside the installed package and holds the trained cascade XML files; cv2.HOGDescriptor\_getDefaultPeopleDetector() returns the trained SVM coefficients.

```
python3 -c "import cv2,
os;p=cv2.data.harcascades+'haarcascade_frontalface_default.xml';print(os.path.
exists(p));h=cv2.HOGDescriptor();h.setSVMDetector(cv2.HOGDescriptor_getDefaultP
eopleDetector());print('HOG detector loaded')"
```

3. Open data/ground-truth.json. Each entry is a frame name mapped to a list of boxes in [x, y, w, h] pixel form with a class label. Count the total number of ground-truth boxes per class and record it — recall is measured against that denominator.

```
python3 -c "import json,collections;g=json.load(open('data/ground-
truth.json'));c=collections.Counter(b['class'] for v in g['frames'].values()
for b in v);print(dict(c))"
```

4. Explain the Viola-Jones cascade in your report before running it, in four parts: Haar-like rectangle features (differences of summed rectangle intensities), the integral image (which makes any rectangle sum a four-lookup operation), AdaBoost (which selects the small subset of features that actually discriminate), and the cascade of stages (which rejects most background windows in the first few stages).

5. Run the cascade with default parameters and record the number of detections per frame. scaleFactor controls the image-pyramid step and minNeighbors controls how many overlapping hits are required before a detection is emitted.

```
cas = cv2.CascadeClassifier(cv2.data.harcascades +
 'haarcascade_frontalface_default.xml')
boxes = cas.detectMultiScale(gray, scaleFactor=1.1, minNeighbors=5,
 minSize=(30, 30))
```

6. Now explain HOG + SVM: the image is divided into cells, a histogram of oriented gradients is accumulated per cell, cells are block-normalised for illumination invariance, and the concatenated descriptor is classified by a linear SVM slid across the image at multiple scales. It is a different family from the cascade — gradient statistics rather than intensity differences.

7. Run the HOG people detector. Note that `detectMultiScale` on `HOGDescriptor` returns boxes AND weights; the weights are the SVM decision values and are what you sweep to build a PR curve.

```
hog = cv2.HOGDescriptor()
hog.setSVMDetector(cv2.HOGDescriptor_getDefaultPeopleDetector())
boxes, weights = hog.detectMultiScale(img, winStride=(8, 8), padding=(8, 8),
 scale=1.05)
```

8. Compare raw and adjusted HOG boxes. The supplied evaluator uses a dataset-specific 15% width and 5% height shrink. It is a post-processing choice, not a universal correction: report it beside metrics, evaluate the raw boxes as a baseline, and do not tune it against a held-out test set. Some boxes may improve and others worsen.

```
x, y, bw, bh = box
pw, ph = int(0.15 * bw), int(0.05 * bh)
corrected = [x + pw, y + ph, bw - 2 * pw, bh - 2 * ph]
```

9. Implement IoU from the definition rather than importing it. For two boxes given as  $[x, y, w, h]$ , the intersection width is  $\max(0, \min(x_1+w_1, x_2+w_2) - \max(x_1, x_2))$  and similarly for height;  $\text{IoU} = \text{intersection area} / (\text{area}_1 + \text{area}_2 - \text{intersection area})$ . Test it on two boxes you can verify by hand.

```
def iou(a, b):
 ax2, ay2 = a[0] + a[2], a[1] + a[3]
 bx2, by2 = b[0] + b[2], b[1] + b[3]
 iw = max(0, min(ax2, bx2) - max(a[0], b[0]))
 ih = max(0, min(ay2, by2) - max(a[1], b[1]))
 inter = iw * ih
 union = a[2] * a[3] + b[2] * b[3] - inter
 return inter / union if union else 0.0
```

10. Hand-verify: box A =  $[0,0,100,100]$  and box B =  $[50,0,100,100]$  overlap on half their width, so intersection =  $50 \times 100 = 5000$ , union =  $10000 + 10000 - 5000 = 15000$ , and  $\text{IoU} = 0.333$ . Confirm your function returns that.

11. Implement greedy matching. Sort predictions by score, and for each prediction take the highest-IoU unmatched ground-truth box of the SAME CLASS at or above the IoU threshold. Matched = true positive; unmatched prediction = false positive; unmatched ground truth = false negative. Each ground-truth box may be matched at most once — that rule is what stops a detector scoring well by emitting the same box ten times.

12. Compute precision =  $TP / (TP + FP)$  and recall =  $TP / (TP + FN)$ , then  $F1 = 2PR / (P + R)$ . Write the three formulas in your report; the assessor asks for them, not just the numbers.

13. Run the evaluator at three IoU thresholds: 0.3 (loose localisation), 0.5 (an example default) and 0.7 (tight). Record whether precision and recall change as the threshold tightens.

```
python3 detect_eval.py --frames reference/frames --truth data/ground-truth.json
--config data/eval-config.json --out out
```

14. Run the separate realistic synthetic positive control with the command below. Keep its metrics separate from the twelve schematic frames. It contains one image, two manually annotated faces and two people; annotations are approximate, not detector-derived. The face detector may find these faces; HOG person detections are not guaranteed. Inspect overlays and report actual class-specific TP/FP/FN. This checks pipeline operation, not deployment quality or population accuracy.

```
python3 detect_eval.py --frames reference/positive-control --truth
data/positive-control-truth.json --config data/eval-config.json --out
out/positive-control
```

15. Sweep `minNeighbors` from 1 to 8 for the cascade and record precision and recall at each value. Lower settings may emit more detections; higher settings may suppress them. Neither precision improvement nor strict monotonicity is guaranteed. Plot or tabulate the pairs.

16. Sweep the HOG SVM weight threshold across the range in `data/eval-config.json` and record the same pairs. This gives you a score-threshold precision/recall table on a fixed HOG candidate pool rather than a single point.

17. Select and state your operating point using the rule in `data/eval-config.json`. The rule is not 'highest F1' by default — read it. For a review-queue application recall matters more than precision, because a missed target never reaches a human at all.

18. Write the honest limitations paragraph. State that these are synthetic schematic targets, that the numbers therefore describe the detectors on THIS distribution only, and that published detector accuracies are dataset-specific. Note as a reference point that published deepfake-detection backbones report validation accuracies in the mid-to-high 70s on hard, shifted data while AI-generated-image classifiers report 99%+ on curated benchmarks — the same metric, very different problems.

19. Save `out/detections.json`, `out/evaluation.csv`, your PR sweep table and the stated operating point, then complete the evidence checklist.

## Verify

`out/evaluation.csv` reports TP, FP, FN, precision, recall and F1 for both detectors at IoU 0.3, 0.5 and 0.7; your hand-computed IoU of 0.333 matches your function; and you

have stated one operating point with the exact parameter values that produce it and the selection rule you applied.

### Expected outputs

- `out/detections.json` — every predicted box with its frame, class and score.
- `out/evaluation.csv` — one row per (detector, IoU threshold) with TP/FP/FN/P/R/F1.
- Observed metrics at all IoU thresholds, including legitimate unchanged values.
- A minNeighbors operating-parameter sweep; report actual values without forcing a trend.
- A HOG score-threshold sweep; precision need not be monotone.
- `out/overlay_frame_00.png` — a frame with ground truth in one colour and predictions in another.

### Evidence checklist

- The four-part explanation of the Viola-Jones cascade in your own words.
- Your IoU function and the hand-verified 0.333 result.
- The precision, recall and F1 formulas written out.
- `out/evaluation.csv` at all three IoU thresholds.
- The parameter sweep table for at least one detector.
- Your stated operating point, the parameter values, and the selection rule you applied.
- The limitations paragraph naming the synthetic distribution. Include `out/positive-control/evaluation.csv` and `overlay`, with separate sample size (one image/two faces/two people) and manual-annotation uncertainty.

### Troubleshooting

| Symptom                                                              | What to do                                                                                                                                                                                                                                                                                                                                                                                                                   |
|----------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| The cascade returns zero detections on every frame                   | Zero detections on schematic targets are a valid distribution-shift finding. Do not invent detections or select a deployable point if no operating point qualifies. As a documented exploratory comparison, lower <code>minNeighbors</code> to 2-3 and <code>minSize</code> to (20,20), and confirm you passed a GRAYSCALE image — <code>detectMultiScale</code> on input conventions should be verified for the chosen API. |
| <code>cv2.data.harcascades</code> raises <code>AttributeError</code> | You are on a very old wheel. <code>pip install --upgrade opencv-python</code> . Do not download cascade XML files from the internet; they ship in the package.                                                                                                                                                                                                                                                               |
| HOG returns an empty weights array                                   | Some builds return an empty tuple when no detection passes. Guard with <code>if len(boxes) == 0: continue</code> before indexing weights.                                                                                                                                                                                                                                                                                    |
| Recall is above 1.0                                                  | You matched one ground-truth box to more than one prediction. Enforce the match-at-most-once rule with a <code>matched</code> set keyed by ground-truth index.                                                                                                                                                                                                                                                               |

| Symptom                                                   | What to do                                                                                                                                                        |
|-----------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Precision and recall are identical at every IoU threshold | This is possible (for example zero detections or all boxes above 0.7). Check that the threshold is passed into the matcher and not hard-coded.                    |
| The two detectors are compared as if they do the same job | They do not: the cascade here targets face-like regions and HOG targets person-like regions. Evaluate each against its own class in the ground truth, and say so. |

**Note:** The same content, plus the full prompt pack, is in [labs/lab-08-object-detection-evaluation/README.md](#) and its PDF. Sources used in this lab: S16, S17, S19, S23.

## Lab 09 — Storyboard to Video Prompt Pack and Deterministic Animatic

**Outcome:** ELO5 · **Abilities:** A6 · **Knowledge:** K10

**Slide:** 158 · **Folder:** [labs/lab-09-storyboard-video-prompting/](#) · **In class:** 30 minutes

### Scenario

A 24-second product film has to be specified before anyone renders anything. Build the shot list, write a continuity bible that pins the things that must not drift between shots, author a structured per-shot video prompt with a per-second audio track, and render a deterministic animatic from your storyboard frames so the timing can be reviewed today.

### What you will produce

[out/shot-list.csv](#), [out/continuity-bible.md](#), [out/video-prompt-pack.json](#) (three shots with per-second audio), and [out/animatic.mp4](#) — a deterministic OpenCV render of the storyboard frames, explicitly labelled as an animatic and NOT as generative-model output.

\*Tools:\* Text editor · Python 3 · opencv-python · OPTIONAL free-tier video model. The lab completes fully offline; no paid service is required.

### Files supplied with this lab

| Path                                                                       | What it is                                                                                                                                       |
|----------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">reference/board/shot1_a.png</a> .. <a href="#">shot3_c.png</a> | Nine 1280x720 storyboard frames, three per shot (deterministically rendered with OpenCV — SIMULATED storyboard art, not generative-model output) |
| <a href="#">data/film-brief.md</a>                                         | The 24-second NorthQuay 'Harbour Line' film brief with the mandatory beats                                                                       |
| <a href="#">data/shot-template.json</a>                                    | The per-shot prompt schema, including the audio track structure                                                                                  |
| <a href="#">data/continuity-fields.json</a>                                | The continuity fields that must be identical across all shots                                                                                    |
| <a href="#">animatic.py</a>                                                | Runnable script                                                                                                                                  |
| <a href="#">PROMPTS.md</a> / <a href="#">PROMPTS.pdf</a>                   | The reusable prompt and specification pack                                                                                                       |

## Step-by-step

1. Use the prepared offline OpenCV environment. Work in this lab folder and create out/. The animatic is silent; audio cues are written planning deliverables only. Save out/shot1.json, shot2.json and shot3.json with continuity as an OBJECT, replacing the example placeholder string. Write out/continuity-bible.md, then combine the header and these objects under a shots array in out/video-prompt-pack.json.

2. Read [data/film-brief.md](#). Extract the mandatory beats, the total duration, and the delivery aspect ratios. Note that the brief specifies 24 seconds — for this exercise using a base generation length of 8 seconds per shot, that is three shots, not one clip.

3. Build the shot list as [out/shot-list.csv](#) with columns: shot, seconds, purpose, subject\_action, camera, location, audio\_summary. Three rows of 8 seconds each. Every row must have a PURPOSE — a shot with no job in the narrative gets cut, not shortened.

4. Write the continuity bible. Open [data/continuity-fields.json](#) and fill every field. These are the properties that must be byte-identical across all three prompts: product description, wardrobe, model description, time of day, weather, lighting direction, colour palette and lens.

5. State the continuity risk for each field in one clause. For example: 'jacket shell colour — highest risk, because the shell occupies a small frame fraction in shot 3 and the model has least evidence to hold it there.' This is what tells the reviewer where to look.

6. Author shot 1's prompt against [data/shot-template.json](#). The schema separates SUBJECT, ACTION, CAMERA, LOCATION, LIGHTING and NEGATIVE — the same factor separation you compared in Lab 03, extended with the two video-only fields: camera MOVE and shot DURATION.

7. Author shots 2 and 3 by copying shot 1 and changing ONLY the fields the shot list says change. Then diff the three JSON files and confirm that every continuity field is character-identical. Different words can increase drift risk; identical text does not guarantee visual continuity.

```
python3 -c "import json;a,b,c=[json.load(open(f'out/shot{i}.json')) for i in (1,2,3)];print([k for k in a['continuity'] if not (a['continuity'][k]==b['continuity'][k]==c['continuity'][k])] or 'CONTINUITY FIELDS IDENTICAL')"
```

8. Add the per-second audio specification to each shot. Selected Veo models can generate audio natively; these per-second cues communicate intent but do not guarantee exact timing. Specify what is heard in each second: ambience, any dialogue with its timing, and any music cue. Write 0-1s, 1-2s ... 7-8s for every shot.

9. Add the negative constraints to every shot. At minimum: no rendered text or logos inside the frame (add those in post so they stay editable and legible), no additional people, no lens flare, and no change to the jacket shell colour.
10. Record the documented generation parameters in the pack header rather than in prose: model id, durationSeconds, aspectRatio and resolution. Verified against the official documentation on 6 September 2026: Veo 3.1 model IDs are veo-3.1-generate-preview, veo-3.1-fast-generate-preview and veo-3.1-lite-generate-preview; durationSeconds accepts 4, 6 or 8; aspectRatio accepts 16:9 (default) or 9:16; resolution accepts 720p, 1080p or 4k with Lite limited to 720p/1080p. 1080p/4k require 8 seconds. Extension is 720p; Lite supports neither referenceImages nor extension. Audio is generated natively. veo-3.0-generate-001 is deprecated.
11. Record the continuity mechanisms the documentation actually offers, so your plan does not depend on hope: on supported non-Lite models, up to three reference images to guide content, image-to-video from a fixed starting frame, and video extension that adds 7 seconds to an existing clip. State which one you would use to hold the product across your three shots, and why.
12. OPTIONAL — if your account has eligible access and quota, submit shot 1 and note that generation is a long-running operation: you submit, then poll until done. Do not block the class on it. If you do not have access, continue; the lab completes without it.

```
Documented long-running-operation pattern (verified 6 Sep 2026):
from google import genai
from google.genai import types
import time, json
from pathlib import Path
Before class install google-genai and set GEMINI_API_KEY privately. Access
may require billing.
client=genai.Client()
prompt_text=json.dumps(json.load(open('out/shot1.json')))
operation = client.models.generate_videos(
 model="veo-3.1-generate-preview", prompt=prompt_text,
 config=types.GenerateVideosConfig(duration_seconds=8, aspect_ratio='16:9',
 resolution='1080p'))
deadline=time.monotonic()+180
while not operation.done and time.monotonic()<deadline:
 time.sleep(10)
 operation = client.operations.get(operation)
if not operation.done: print('Still pending; record operation.name and continue
offline')
elif operation.error: print(operation.error)
elif operation.response and operation.response.generated_videos:
 video=operation.response.generated_videos[0].video
 client.files.download(file=video)
 video.save('out/shot1-generated.mp4')
else: print('No video returned; record status and continue offline')
```

13. Now build the animatic. Load the nine storyboard frames and render them to `out/animatic.mp4` at the shot-list timings, with a burned-in shot label, a running timecode and a persistent ANIMATIC banner.

```
python3 animatic.py --board reference/board --shots out/shot-list.csv --out out/animatic.mp4
```

14. Verify the render: open it, confirm the duration matches the shot list total, and confirm the ANIMATIC banner is present on every frame.

```
python3 -c "import cv2;c=cv2.VideoCapture('out/animatic.mp4');n=int(c.get(cv2.CAP_PROP_FRAME_COUNT));f=c.get(cv2.CAP_PROP_FPS);print(n,'frames',f,'fps','%.2fs'%(n/f))"
```

15. Write the provenance line for the animatic and put it in your submission verbatim: `'out/animatic.mp4 is a DETERMINISTIC ANIMATIC rendered from storyboard frames with OpenCV. It is not generative-model output and must not be presented as such.'` This matters: mislabelling a deterministic render as model output is a governance failure, not a presentation choice.

16. Review the animatic against the brief's beats. Is 8 seconds enough for shot 2's action? Adjust the action within the fixed 8-second shots, re-render, and record what you changed — that iteration is the entire value of an animatic.

17. Save `out/shot-list.csv`, `out/continuity-bible.md`, `out/video-prompt-pack.json`, `out/animatic.mp4` and the provenance line, then complete the evidence checklist.

## Verify

The three shot JSON files have character-identical continuity blocks, every shot carries eight per-second audio entries and a negative-constraint list, `out/animatic.mp4` plays for the shot-list total duration with a visible ANIMATIC banner, and your submission carries the verbatim provenance line.

## Expected outputs

- `out/shot-list.csv` — 3 rows totalling 24 seconds, each with a stated purpose.
- `out/continuity-bible.md` — every field from `data/continuity-fields.json` filled, each with a risk clause.
- `out/video-prompt-pack.json` — 3 shots, identical continuity blocks, 8 audio entries per shot, negative constraints on every shot.
- A documented-parameters header with model id, durationSeconds, aspectRatio, resolution.
- `out/animatic.mp4` — 24 seconds, shot labels, running timecode, persistent ANIMATIC banner.
- The verbatim provenance line for the animatic.

## Evidence checklist

- `out/shot-list.csv` with the purpose column filled for all three shots.
- The continuity diff output showing CONTINUITY FIELDS IDENTICAL.
- One shot's full per-second audio specification.
- The negative constraints list used on every shot.
- The documented parameter header, with the note that `veo-3.0-generate-001` is deprecated.
- Which continuity mechanism you chose (reference images, image-to-video, or extension) and why.
- `out/animatic.mp4` duration check output.
- The verbatim animatic provenance line.

## Troubleshooting

| Symptom                                                   | What to do                                                                                                                                                                                                                                  |
|-----------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| The continuity diff reports differing fields              | That is the lab working. Copy the continuity block from shot 1 into shots 2 and 3 programmatically rather than retyping it — retyping is exactly how drift enters.                                                                          |
| <code>out/animatic.mp4</code> is 0 bytes or will not open | The fourcc is unsupported on your build. Try <code>cv2.VideoWriter_fourcc(*'mp4v')</code> , and confirm every frame you write has the same shape as the writer's declared size — a single mismatched frame silently produces an empty file. |
| The animatic runs at the wrong speed                      | The writer FPS and your frame-repeat count must agree: at 24 fps an 8-second shot needs 192 written frames. Compute the repeat count from the shot list, do not hard-code it.                                                               |
| A storyboard frame is a different size from the others    | Resize every frame to the writer size on load with <code>cv2.resize</code> . Mixed sizes are the most common cause of a truncated video.                                                                                                    |
| You want to submit the animatic as the finished film      | Do not. It is a timing artefact. Its purpose is to let the brief be corrected before any generation budget is spent, and it must stay labelled.                                                                                             |
| A free-tier video request never completes                 | Check operation status/error, quota and account access; no free-tier availability is assumed. Use bounded polling, then move on — the deliverable is the prompt pack and the animatic, not the rendered film.                               |

**Note:** The same content, plus the full prompt pack, is in [labs/lab-09-storyboard-video-prompting/README.md](#) and its PDF. Sources used in this lab: S10, S11, S22.

## Lab 10 — Motion, Tracking and Temporal Continuity Measurement

**Outcome:** ELO5 · **Abilities:** A6 · **Knowledge:** K10

**Slide:** 159 · **Folder:** [labs/lab-10-motion-tracking-continuity/](#) · **In class:** 60 minutes

## Scenario

A delivered clip 'feels jittery' and 'the product seems to drift'. Turn both complaints into numbers. Isolate the moving foreground with background subtraction, track the subject with mean shift, CAMShift and a learning tracker, measure the dense motion field with optical flow, and report centroid jitter, scale drift and inter-frame structural similarity.

### What you will produce

out/tracking.csv with per-frame centroid and box size for three trackers,  
out/continuity-report.json with jitter, drift and inter-frame SSIM statistics, and  
annotated overlay videos for each tracker.

\*Tools:\* Python 3 · opencv-python · NumPy · no network, no paid service

### Files supplied with this lab

| Path                            | What it is                                                                                                                                                                                                                                                                  |
|---------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| reference/nq-motion-clip.mp4    | Synthetic 8-second 640x480 25 fps clip: a subject block translates across a static shelf background while growing in scale, with a deliberate two-frame position discontinuity at t=4.0 s (deterministically rendered with OpenCV — SIMULATED, not generative-model output) |
| reference/nq-motion-truth.json  | The exact per-frame subject box used to render the clip, so every tracking error is measurable rather than estimated                                                                                                                                                        |
| data/tracking-config.json       | The initial track window, the tracker list, and the continuity thresholds                                                                                                                                                                                                   |
| data/continuity-thresholds.json | Pass/fail limits for centroid jitter, scale drift and inter-frame SSIM                                                                                                                                                                                                      |
| motion.py                       | Runnable script                                                                                                                                                                                                                                                             |
| PROMPTS.md / PROMPTS.pdf        | The reusable prompt and specification pack                                                                                                                                                                                                                                  |

### Step-by-step

1. Open the clip and confirm its properties before anything else: 640x480, 25 fps, 200 decodable frames, 8.00 s. Lab 02 taught you not to trust the container.

```
python3 -c "import cv2;c=cv2.VideoCapture('reference/nq-motion-clip.mp4');print(int(c.get(3)),int(c.get(4)),c.get(5),int(c.get(7)))"
```

2. Run background subtraction with `cv2.createBackgroundSubtractorMOG2()`. MOG2 models each pixel as a mixture of Gaussians over time and labels a pixel foreground when it no longer fits its learned model. Report the foreground pixel count per frame.

```
sub = cv2.createBackgroundSubtractorMOG2(history=200, varThreshold=16,
 detectShadows=True)
```

```
fg = sub.apply(frame)
```

3. Note that `detectShadows=True` labels shadow pixels 127 rather than 255. Threshold at 200 to get a hard foreground mask, or you will count shadows as subject.

```
_, hard = cv2.threshold(fg, 200, 255, cv2.THRESH_BINARY)
```

4. Repeat with `cv2.createBackgroundSubtractorKNN()` and compare the foreground counts over the first 50 frames. Record which stabilises faster — the model needs

history before it is reliable, which is why the first frames of any BS pipeline are untrustworthy.

5. Explain background subtraction in one paragraph for your report: it builds a per-pixel model of the static scene from recent history and flags pixels that deviate from it, producing a binary foreground mask. It requires a static camera; a pan invalidates the model.

6. Set up mean shift. Take the initial window from `data/tracking-config.json`, compute the hue histogram of that region, and back-project it onto each frame. Mean shift then climbs the density of the back-projection to find the window's new position.

```
roi_hsv = cv2.cvtColor(roi, cv2.COLOR_BGR2HSV)
mask = cv2.inRange(roi_hsv, (0, 60, 32), (180, 255, 255))
hist = cv2.calcHist([roi_hsv], [0], mask, [180], [0, 180])
cv2.normalize(hist, hist, 0, 255, cv2.NORM_MINMAX)
```

7. Run `cv2.meanShift(back_projection, window, term_crit)` frame by frame and record the window each time. Note the defining limitation: the window size never changes, so as the subject grows the window stops containing it.

```
term = (cv2.TERM_CRITERIA_EPS | cv2.TERM_CRITERIA_COUNT, 10, 1)
ret, window = cv2.meanShift(bp, window, term)
```

8. Run `cv2.CamShift` with the identical setup. CAMShift re-computes the window size from the zeroth moment of the back-projection each iteration and also returns an orientation, so it adapts to a subject that changes scale.

```
rot_rect, window = cv2.CamShift(bp, window, term)
```

9. Quantify the difference. The clip's subject grows by a known factor between the first and last frame — read it from `reference/nq-motion-truth.json`. Report each tracker's final box area as a ratio of its initial area, and compare against the truth ratio. This is the concrete evidence for when to choose CAMShift over mean shift.

10. Run the third tracker, `cv2.TrackerMIL_create()`. Note the API-currency point: `cv2.TrackerCSRT_create` and `cv2.TrackerKCF_create` are NOT in the base `opencv-python` wheel — they require `opencv-contrib-python`. TrackerMIL is built in, which is why this lab uses it.

```
tracker = cv2.TrackerMIL_create()
tracker.init(first_frame, tuple(init_window))
ok, box = tracker.update(frame)
```

11. Run all three trackers over the clip and write `out/tracking.csv` with, per frame and per tracker: centre x, centre y, width, height, and the IoU against the ground-truth box.

```
python3 motion.py --video reference/nq-motion-clip.mp4 --truth reference/nq-motion-truth.json --config data/tracking-config.json --out out
```

12. Compute dense optical flow with `cv2.calcOpticalFlowFarneback` between consecutive grayscale frames. The result is an H x W x 2 array of per-pixel (dx, dy) displacement. Convert to magnitude and angle with `cv2.cartToPolar` and report the mean magnitude per frame.

```
flow = cv2.calcOpticalFlowFarneback(prev_gray, gray, None,
 0.5, 3, 15, 3, 5, 1.2, 0)
mag, ang = cv2.cartToPolar(flow[..., 0], flow[..., 1])
```

13. Find the discontinuity. The clip contains a deliberate two-frame position jump at  $t = 4.0$  s (frame 100). Inspect whether it appears as a spike in mean flow magnitude and as a spike in centroid step size. These are candidate flags, not guaranteed detectors; inspect the clip and tracker IoU to explain any missed jump.

14. Compute the three continuity metrics. CENTROID JITTER: the standard deviation of the frame-to-frame centroid step, excluding only explicitly declared transition indices in `config.excluded_transition_frames`. SCALE DRIFT: the absolute difference between the tracker's area ratio and the ground truth's. INTER-FRAME SSIM: the mean SSIM between consecutive frames, which falls sharply at a cut or flicker in some clips; static backgrounds can dominate this score.

15. Reuse the SSIM implementation from Lab 05 rather than writing a second one — a shared copy of `ssim.py` is included in this lab folder for self-containment. Confirm it is byte-identical to the Lab 05 copy so the two labs cannot disagree.

```
python3 -c "import
hashlib;print(hashlib.sha256(open('ssim.py','rb').read()).hexdigest()[:16])"
```

16. Read `data/continuity-thresholds.json` and mark each metric PASS or FAIL. Report the jitter/drift verdict per tracker and SSIM once for the source clip. Tracking errors and real clip defects can both affect results; use truth IoU and visual review to distinguish them.

17. Write the interpretation paragraph. Connect it back to generative video: temporal coherence is the property that distinguishes a video model from a per-frame image model, and it is why architectures extend the 2-D U-Net to 3-D or treat video as a sequence of spatiotemporal patches. Your jitter and inter-frame SSIM numbers are the simple diagnostic proxies. FVD instead compares distributions of learned video features; these scores neither compute FVD nor certify semantic coherence.

18. Save `out/tracking.csv`, `out/continuity-report.json` and the three overlay videos, then complete the evidence checklist.

## Verify

`out/tracking.csv` has a row per frame for all three trackers with an IoU against ground truth, `out/continuity-report.json` reports centroid jitter, scale drift and clip-level mean inter-frame SSIM, plus per-tracker jitter/drift PASS/FAIL, and you have located the frame-100 discontinuity in both the optical-flow magnitude and the centroid step series.

## Expected outputs

- `out/tracking.csv` — 200 frames x 3 trackers with `cx`, `cy`, `w`, `h` and `iou_vs_truth`.
- Mean shift final/initial box area ratio  $\sim 1.0$  (the window never resizes).

- CAMShift area ratio tracking the ground-truth growth more closely if tracking remains valid; verify rather than assume.
- Inspect flow/centroid candidates around frame 100; large displacements may violate flow assumptions or be diluted by static background.
- `out/continuity-report.json` — jitter/scale drift per tracker and clip-level inter-frame SSIM with PASS/FAIL against the thresholds.
- `out/track_meanshift.mp4`, `out/track_camshift.mp4`, `out/track_mil.mp4` overlays.

### Evidence checklist

- `out/continuity-report.json` in full.
- The mean shift and CAMShift area ratios next to the ground-truth ratio.
- The frame index of the detected discontinuity, from both the flow and centroid series.
- Mean IoU against ground truth for each of the three trackers.
- One paragraph explaining background subtraction and its static-camera requirement.
- The API-currency note: which tracker constructors are absent from the base wheel.
- The interpretation paragraph linking your metrics to temporal coherence in video models.

### Troubleshooting

| Symptom                                                                | What to do                                                                                                                                                                                                                                                       |
|------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>cv2.TrackerCSRT_create</code> raises <code>AttributeError</code> | Expected on the base <code>opencv-python</code> wheel — CSRT and KCF live in <code>opencv-contrib-python</code> . Use <code>cv2.TrackerMIL_create()</code> , which is built in. Do not add a contrib dependency for this lab.                                    |
| Mean shift locks onto the background                                   | The initial window includes too much background, so the hue histogram is dominated by it. Shrink the initial window to the subject, and keep the <code>cv2.inRange(..., (0,60,32), (180,255,255))</code> saturation/value floor that excludes washed-out pixels. |
| CAMShift collapses the window to a few pixels                          | The back-projection has almost no support — usually because the subject left the frame or changed hue. Re-seed from the ground-truth box and report the frame where it collapsed as a finding.                                                                   |
| Optical flow magnitude is enormous everywhere                          | You passed colour frames. <code>calcOpticalFlowFarneback</code> requires single-channel 8-bit input; convert both frames with <code>cv2.COLOR_BGR2GRAY</code> first.                                                                                             |
| Inter-frame SSIM is 1.0 for every pair                                 | You are comparing a frame with itself — check the previous-frame variable is actually being updated at the end of the loop.                                                                                                                                      |
| The discontinuity does not show in the centroid series                 | Your tracker lost the subject before frame 100, so there is no step to measure. Check the IoU column: once it hits zero the series after it is meaningless, and you must say so rather than reporting the numbers.                                               |
| Overlay videos are empty                                               | Same cause as Lab 09: a frame written to the <code>VideoWriter</code> with a different shape from the declared size. Resize every frame before writing.                                                                                                          |

**Note:** The same content, plus the full prompt pack, is in [labs/lab-10-motion-tracking-continuity/README.md](#) and its PDF. Sources used in this lab: S10, S17, S23.

## Lab 11 — Captions, Overlays, Safe Areas and Export Profiles

**Outcome:** ELO5 · **Abilities:** A6 · **Knowledge:** K10

**Slide:** 160 · **Folder:** [labs/lab-11-captions-overlays-export/](#) · **In class:** 30 minutes

### Scenario

The film is approved. Now it has to ship in three aspect ratios with burned-in captions that are legible on a phone, a logo that stays inside the safe area in every crop, and a file size that survives the platform's limit. Build the export ladder and measure every constraint rather than eyeballing it.

### What you will produce

Three exported renditions (16:9, 1:1, 9:16) with burned-in captions and overlay, [out/export-report.csv](#) with per-rendition dimensions, duration, file size and bit-rate, and [out/safe-area-check.csv](#) proving the caption and logo stay inside the safe area in all three.

**\*Tools:** Python 3 · opencv-python · NumPy · no network, no paid service; silent fixture (OpenCV export does not preserve audio)

### Files supplied with this lab

| Path                                                     | What it is                                                                                                                           |
|----------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">reference/nq-approved-cut.mp4</a>            | Synthetic 12-second 1920x1080 25 fps approved cut (deterministically rendered with OpenCV — SIMULATED, not generative-model output)  |
| <a href="#">reference/nq-logo-overlay.png</a>            | 240x120 logo overlay with an alpha channel (deterministically rendered — SIMULATED)                                                  |
| <a href="#">data/captions.json</a>                       | The caption cue list: start, end, text and speaker for each cue                                                                      |
| <a href="#">data/export-profiles.json</a>                | The three delivery profiles with target dimensions, safe-area insets, maximum file size and minimum caption H-glyph height in pixels |
| <a href="#">export_ladder.py</a>                         | Runnable script                                                                                                                      |
| <a href="#">PROMPTS.md</a> / <a href="#">PROMPTS.pdf</a> | The reusable prompt and specification pack                                                                                           |

### Step-by-step

1. Read [data/captions.json](#). Each cue has start and end times in seconds and the caption text. Confirm the cues do not overlap and that the last cue ends within the clip duration — this script rejects overlaps instead of silently selecting one cue.

```
python3 -c "import json;c=json.load(open('data/captions.json'))
['cues'];print(len(c),'cues, overlaps:',[i for i in range(1,len(c)) if c[i]
['start']<c[i-1]['end']] or 'none']"
```

2. Read `data/export-profiles.json`. Each profile gives the output dimensions, the safe-area inset as a fraction, the maximum file size in bytes, and the minimum caption H-glyph height in pixels. That last one is the legibility constraint: a caption scaled down with the frame becomes unreadable on a phone.

3. Compute the required font scale per profile rather than hard-coding it. Use `cv2.getTextSize` to measure the rendered height at scale 1.0, then solve for the scale that reaches the profile's minimum H-glyph height.

```
(tw, th), base = cv2.getTextSize('Hg', cv2.FONT_HERSHEY_SIMPLEX, 1.0, 2)
scale = required_cap_height_px / float(th)
```

4. Draw a caption with a background plate rather than plain text. Measure the text, draw a filled rectangle with a few pixels of padding, then draw the text over it. Text without a plate is unreadable over a bright frame and there is no way to fix it later.

```
cv2.rectangle(frame, (x - 8, y - th - 8), (x + tw + 8, y + base + 4), (18, 18, 22), -1)
cv2.putText(frame, text, (x, y), cv2.FONT_HERSHEY_SIMPLEX, scale,
 (245, 245, 245), 2, cv2.LINE_AA)
```

5. Define the safe area numerically: inset the frame by the profile's fraction on all four sides. Every caption plate and the logo must lie entirely inside it. Compute the safe rectangle in pixels for each profile and record it.

6. Composite the logo using its alpha channel, not a rectangular paste. Read the PNG with `cv2.IMREAD_UNCHANGED` to get four channels, split off the alpha, normalise it to [0,1] and blend.

```
logo = cv2.imread('reference/nq-logo-overlay.png', cv2.IMREAD_UNCHANGED)
bgr, alpha = logo[:, :, :3], logo[:, :, 3:4].astype(np.float32) / 255.0
roi[:] = (alpha * bgr + (1 - alpha) * roi).astype(np.uint8)
```

7. Decide the reframing rule for each aspect ratio and write it down before coding it. For 1:1 and 9:16 you must either CROP (loses the sides) or LETTERBOX (keeps everything, adds bars). State which you chose per profile and why — this is a content decision, not a technical default.

8. Implement centre-crop-then-scale for the profiles where you chose crop. Compute the source rectangle with the target aspect ratio, crop it, then resize with `cv2.INTER_AREA` — the correct interpolation for downscaling.

```
src_ar = w / h
dst_ar = tw_ / th_
if src_ar > dst_ar: # source is wider: crop the sides
 new_w = int(h * dst_ar); x0 = (w - new_w) // 2
 crop = frame[:, x0:x0 + new_w]
else: # source is taller: crop top and bottom
 new_h = int(w / dst_ar); y0 = (h - new_h) // 2
 crop = frame[y0:y0 + new_h, :]
```

9. Implement letterbox for the profiles where you chose it: scale to fit inside the target and pad the remainder with `cv2.copyMakeBorder`. Record the pad size — a 9:16

letterbox of a 16:9 source is mostly bars, which is usually the argument for cropping instead.

10. Render all three profiles.

```
python3 export_ladder.py --video reference/nq-approved-cut.mp4 --captions
data/captions.json --profiles data/export-profiles.json --logo reference/nq-
logo-overlay.png --out out
```

11. Verify the safe area programmatically. For each profile and each caption cue, compute the caption plate rectangle and the logo rectangle in output pixels, and test that both lie inside the safe rectangle. Write `out/safe-area-check.csv` with a PASS/FAIL per cue per profile.

12. Verify legibility. Report the achieved caption H-glyph height in pixels for each profile and confirm it meets the profile minimum. Then express it as a percentage of frame height as a layout proxy; inspect on a phone because reading speed, viewing distance and typography also matter.

13. Verify contrast. Record the relative-sRGB luminance contrast ratio of the chosen solid text and plate colours, using  $(\text{lighter}+0.05)/(\text{darker}+0.05)$ . This is a design-colour check; inspect decoded frames for compression and antialiasing effects.

14. Measure the exports. For each rendition record dimensions, decoded frame count, duration, file size in bytes, and the average container bit-rate in bits per second. Compare each against the profile's maximum file size and mark PASS or FAIL.

15. Where a profile fails on size, state the lever you would pull and why — resolution, frame rate, or encoder quality — and note the trade-off each carries. Do not silently re-encode until it fits and report only the passing run.

16. Confirm every rendition still carries the correct provenance. If the source cut is a simulated classroom asset, all three exports are too; the label travels with the derivative. Record this explicitly.

17. Save the three renditions, `out/export-report.csv` and `out/safe-area-check.csv`, then complete the evidence checklist.

## Verify

Three renditions exist at the profile dimensions with the same duration as the source, `out/safe-area-check.csv` shows PASS for every cue in every profile, the achieved caption H-glyph height meets each profile's minimum, and `out/export-report.csv` records size and bit-rate with a PASS/FAIL against each profile's limit.

## Expected outputs

- `out/export_16x9.mp4` at 1920x1080, `out/export_1x1.mp4` at 1080x1080, `out/export_9x16.mp4` at 1080x1920.

- All three the same duration as the source (12.00 s).
- `out/safe-area-check.csv` — one row per (profile, cue) plus a logo row, all PASS.
- Achieved caption H-glyph height at or above the profile minimum in all three.
- `out/export-report.csv` — dimensions, frames, duration, bytes, bits per second, size verdict.
- A stated crop-or-letterbox decision with a reason for each non-16:9 profile.

### Evidence checklist

- `out/export-report.csv` in full.
- `out/safe-area-check.csv` showing every cue inside the safe rectangle.
- The computed font scale per profile and the achieved H-glyph height in pixels and as a percentage of frame height.
- The computed design-colour contrast ratio and a decoded-frame visual check.
- Your crop-versus-letterbox decision per profile, with the reason.
- One still frame from each rendition showing the caption and logo in place.
- The provenance statement confirming the exports inherit the source's SIMULATED label.

### Troubleshooting

| Symptom                                    | What to do                                                                                                                                                                                                                                    |
|--------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Captions are cut off at the frame edge     | You positioned the text by a fixed pixel offset that does not scale with the profile. Compute the caption origin from the safe rectangle, never from a constant.                                                                              |
| The logo has a black box around it         | You read the PNG without <code>cv2.IMREAD_UNCHANGED</code> , so the alpha channel was dropped. Read with four channels and blend using alpha.                                                                                                 |
| <code>cv2.putText</code> renders nothing   | The font scale computed from <code>getTextSize</code> can come out near zero if you divided by the wrong element. <code>getTextSize</code> returns ((width, height), baseline) — the H-glyph height is the second element of the first tuple. |
| The 9:16 export is mostly empty bars       | You letterboxed a 16:9 source into a 9:16 frame. That is arithmetically correct and usually the wrong choice — crop instead, and state the loss.                                                                                              |
| Output duration is shorter than the source | Frames were dropped because a resized frame did not match the writer's declared size. Assert <code>frame.shape[:2] == (out_h, out_w)</code> before every write.                                                                               |
| File size fails the profile limit          | Record the failure and the lever you would pull. <code>cv2.VideoWriter</code> gives limited rate control; note that a production pipeline would hand off to a dedicated encoder, and say so rather than pretending the constraint was met.    |

**Note:** The same content, plus the full prompt pack, is in `labs/lab-11-captions-overlays-export/README.md` and its PDF. Sources used in this lab: S11, S12, S23.

## Topic 05 — Evaluating Cloud and Edge AI Creative Workflows

LU6 · ELO6 · knowledge K11, K12, K13 · abilities A7, A8

Slides 162–194 · 75 minutes classroom facilitation · 60 minutes practical

Vision system architecture — the four-layer sensors-to-synthesis stack · vision communication protocols · real-world design constraints — latency, bandwidth, privacy, cost and energy · measured cloud-versus-edge trade-offs

### Key concepts

**A generative visual system has four layers.** IoT sensing (cameras, drones, wearables) -> edge intelligence (pre-processing, lightweight inference, federated learning) -> generative visual modelling (VAE/GAN/diffusion, split computing) -> application (dashboards, AR/VR, digital twins).

**Split computing is the real architecture decision.** Latency-critical stages run at the edge; training and heavy sampling run in the cloud. You choose the split point by measuring per-stage latency and the payload size that crosses it.

**Protocols are chosen by payload and latency class.** MQTT and CoAP for small telemetry and events, RTSP/WebRTC for live video, HTTPS/REST and gRPC for model calls, S3-style object storage for renders. Illustrative payload-only example: 100 kB/frame x 10 fps = 1000 kB/s, versus 1 kB/event x 1 event/s = 1 kB/s, a 1000:1 ratio before protocol overhead. Measure actual encoded sizes and event rates before choosing an architecture.

**Sampling steps and frames influence compute cost.** At fixed model, resolution and hardware,  $S \times F$  is a rough workload heuristic. Measured latency also depends on temporal attention, batching and scheduling. Distillation can reduce sampling steps; latent modelling reduces spatial representation cost, not necessarily the frame count.

**Edge fits through quantisation plus adapters.** Moving weights from 32-bit to b-bit scales parameter memory by  $b/32$ ; low-rank adapters train only  $r(d+m)$  parameters instead of  $d \times m$ . Keep sensitive activations at higher precision for stability.

**Energy and carbon are reportable engineering numbers.** Energy per sample is roughly the datacentre overhead factor times the energy per FLOP times the FLOPs per sample. Report hardware, kWh, overhead, grid carbon intensity and joules per sample alongside quality.

### Lab 12 — Cloud-Edge Architecture Design and Measured Trade-offs

**Outcome:** ELO6 · **Abilities:** A7, A8 · **Knowledge:** K11, K12, K13

**Slide:** 190 · **Folder:** [labs/lab-12-cloud-edge-architecture-tradeoffs/](#) · **In class:** 60 minutes

### Scenario

NorthQuay wants the whole creative pipeline running across 41 stores and one cloud account. Measure the real per-stage latency of the representative local operations drawn from Labs 02-11 on this machine, compute the payload that would cross each candidate split point, choose the protocols, and defend one split with numbers rather than preference.

### What you will produce

`out/stage-latency.csv` (measured, not estimated), `out/bandwidth-model.csv` for four upload strategies at 41 stores, `out/split-decision.md` naming the chosen split point with its latency, bandwidth, privacy and cost consequences, and a labelled four-layer architecture diagram.

\*Tools:\* Python 3 · opencv-python · NumPy · offline with prepared dependencies; no paid service

### Files supplied with this lab

| Path                                               | What it is                                                                                                                                                       |
|----------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>reference/nq-pipeline-clip.mp4</code>        | Synthetic 10-second 1280x720 25 fps clip used as the pipeline input for timing (deterministically rendered with OpenCV — SIMULATED, not generative-model output) |
| <code>data/deployment-constraints.json</code>      | Store count, link speeds, privacy classification, latency budget per use case, and the indicative unit costs used for the cost model                             |
| <code>data/protocol-catalogue.json</code>          | Candidate protocols with their payload class, typical latency class and transport, for the protocol-selection step                                               |
| <code>data/split-candidates.json</code>            | The four candidate split points, each with the stage boundary it cuts                                                                                            |
| <code>pipeline_bench.py</code>                     | Runnable script                                                                                                                                                  |
| <code>PROMPTS.md</code> / <code>PROMPTS.pdf</code> | The reusable prompt and specification pack                                                                                                                       |

### Step-by-step

1. Work inside this lab folder with a prepared Python/OpenCV/NumPy environment; create `out/`. This local benchmark measures representative classical stages, not every previous lab or a deployed cloud system. Assume one camera per store unless `cameras_per_store` is explicitly set. Read `split-candidates.json` and map its boundaries to the four payload models; a split is a hypothetical architecture, not an executable cloud deployment. Record all additional assumptions in `out/split-decision.md`.

2. Read `data/deployment-constraints.json`. Note the four numbers that will drive every decision: 41 stores, the per-store uplink in Mbit/s, the latency budget for the shelf-alert use case, and the privacy classification of the camera feed.

3. Draw the four-layer architecture before measuring anything. Layer 1 IoT SENSING: the store cameras and the catalogue capture rig. Layer 2 EDGE INTELLIGENCE: pre-processing, lightweight inference and any federated update. Layer 3 GENERATIVE

VISUAL MODELLING: the generation and heavy editing stages, wherever they run.  
Layer 4 APPLICATION: the review queue, the asset store and the dashboards.

4. Mark on your diagram which of your Labs 02-11 stages sits in which layer. Decode, colour convert and background subtraction are layer 2; detection and tracking are layer 2 or 3 depending on model size; generation, inpainting and outpainting are layer 3; the consistency gate and the review queue are layer 4.

5. Now measure. Run the timing harness, which executes the real pipeline stages on the supplied clip and reports per-stage milliseconds per frame over the whole clip, with a median and a 95th percentile rather than a single average.

```
python3 pipeline_bench.py --video reference/nq-pipeline-clip.mp4 --constraints data/deployment-constraints.json --out out
```

6. Read `out/stage-latency.csv`. Identify the single most expensive stage. Note that the expensive stage on this machine may not be the expensive stage on a store device — record the CPU this was measured on, because a latency number without its hardware is not a measurement.

7. Read the directly timed harness end-to-end median/p95 row; the sum of isolated stage medians is only a rough estimate and is not an end-to-end median, and compare it against the shelf-alert latency budget. State whether the budget is met on this hardware, and by what margin.

8. Model the payload at each split point. Split A sends RAW FRAMES to the cloud: 1280 x 720 x 3 bytes x 25 fps = 69.1 MB/s uncompressed per camera. Split B sends ENCODED VIDEO. Split C sends FEATURE VECTORS. Split D sends EVENTS ONLY — a JSON record when something happens.

9. Compute each payload honestly. For split C use the descriptor sizes you measured in Lab 06: an ORB descriptor is 32 bytes and a default float32 SIFT descriptor is 512 bytes; keypoint coordinates/scale/orientation and framing add overhead, so 500 ORB keypoints per frame is 16 kB per frame, or 400 kB/s at 25 fps. For split D assume the event rate in the constraints file, at roughly 400 bytes per event.

10. Scale to 41 stores and convert to the units the network team uses. Report each split in Mbit/s per store and aggregate Gbit/s across the estate, then compare against the per-store uplink. Check whether any split is arithmetically impossible on the stated uplink — identify it and say so.

11. Compute the monthly transferred data volume for each split using the operating hours in the constraints file, and apply the indicative per-GB cost. Label this clearly as an ILLUSTRATIVE cost model built on the stated unit rates, not a quotation.

12. Select protocols per link from `data/protocol-catalogue.json`. Camera to edge node: RTSP or WebRTC for a live stream. Edge node to cloud for telemetry and events: MQTT (publish/subscribe) or CoAP (request/response, with optional Observe) for

constrained links. Edge or application to a model endpoint: HTTPS/REST or gRPC.  
Rendered assets to storage: object storage over HTTPS.

13. Justify each protocol choice by PAYLOAD CLASS and LATENCY CLASS, not by familiarity. Write one line per link: 'link, payload class, latency class, protocol, reason.' A protocol chosen without reference to payload size is a guess.

14. Apply the generative-latency model. For fixed architecture, resolution and hardware, approximate sampling work as proportional to the number of denoising steps  $S$ ; video sampling latency scales with  $S \times F$  for  $F$  frames. Using the step counts in the constraints file, compute the relative cost of a 25-step image against a 25-step, 200-frame clip — this simplified ratio alone cannot determine whether video generation can run at the edge in this design.

15. The  $S$  times  $F$  model assumes constant per-frame step cost and ignores temporal attention, batching and hardware effects. Read generation-model.csv as an illustrative scaling scenario, not a latency prediction. Weight memory excludes activations, caches, runtime and quantisation metadata. Feature payload excludes metadata unless metadata\_bytes\_per\_keypoint is supplied; sending descriptors is not automatically privacy-preserving. Store-to-cloud transfer is not necessarily billed as cloud egress; verify direction and unit-rate basis before any real quote.

16. Apply the edge-fit levers. Compute the parameter memory of the stated model at 32-bit and at 8-bit: memory scales by  $b/32$ , so 8-bit is a quarter. Then compute the trainable parameter count for a low-rank adapter of rank  $r$  on a  $d \times m$  layer:  $r(d+m)$  against  $d \times m$ . Put both numbers in your decision.

17. Estimate energy per sample as overhead factor  $\times$  energy per FLOP  $\times$  FLOPs per sample using the values in the constraints file, and state what you would have to report for a real carbon figure: hardware inventory, total kWh, the overhead factor, grid carbon intensity and joules per sample. Mark your figure ILLUSTRATIVE.

18. Apply the deployment decision matrix to each eligible selected use case from Lab 01: low-latency image work suggests a distilled sampler with INT8 weights; an edge device suggests INT8 plus parameter-efficient adapters; high resolution suggests latent-space modelling with a few-step sampler; short video suggests temporal attention with distilled steps; brand and style control suggests retrieval conditioning with filters; and a tight privacy constraint suggests a local open model with provenance tracking.

19. Handle privacy explicitly. The camera feed is classified in the constraints file. State which split points are ELIMINATED by that classification before any latency or cost argument is made — a privacy constraint is a gate, not a weighting.

20. Write out/split-decision.md. Name your chosen split point. Give its measured local-harness latency (not split-specific remote latency), its computed bandwidth per store and across the estate, its illustrative monthly cost, the protocols on each link, and

the privacy argument. Then name the runner-up and state the single number that decided between them.

21. Write the residual-risk section. Name at least three: the measurement was taken on developer hardware and not on the store device; the event rate is an assumption; and the model may be replaced, which changes the latency and memory terms. State how each would be closed before build.

22. Save `out/stage-latency.csv`, `out/bandwidth-model.csv`, `out/split-decision.md` and your labelled architecture diagram, then complete the evidence checklist.

## Verify

`out/stage-latency.csv` contains measured median and p95 milliseconds for every pipeline stage with the CPU recorded, `out/bandwidth-model.csv` gives Mbit/s per store and Gbit/s across 41 stores for all four splits and flags any that exceed the stated uplink, and `out/split-decision.md` names one split with its latency, bandwidth, cost, protocols, privacy argument and the deciding number against the runner-up.

## Expected outputs

- Report latency separately from throughput: compare measured serial local-harness milliseconds/frame with 1000/FPS. Even if a frame meets the alert latency budget, processing slower than its arrival interval causes queue growth unless frames are dropped, processing is parallelised or stages are reduced. Results are hardware-dependent.
- `out/stage-latency.csv` — one row per stage: median ms, p95 ms, frames measured, plus the CPU/architecture string and OpenCV version in `environment.json`; `generation-model.csv` is a separate assumed-cost table.
- A directly timed local-harness end-to-end median/p95 against the budget, excluding network/cloud/queue delays.
- `out/bandwidth-model.csv` — four splits x (bytes/s per camera, Mbit/s per store, Gbit/s at 41 stores, monthly GB, illustrative monthly cost, feasible on uplink).
- Raw-frame split feasibility calculated from the actual input and stated uplink, not forced.
- `out/split-decision.md` with the chosen split, the runner-up and the deciding number.
- A labelled four-layer architecture diagram with every Lab 02-11 stage placed.
- The INT8 memory ratio (0.25) and the LoRA trainable-parameter comparison.

## Evidence checklist

- Measured service time versus frame-arrival interval (1000/FPS), the implied serial throughput 1000/service\_ms, and a queue-control plan if arrival rate exceeds processing capacity; no sustained-rate claim from latency alone.
- `out/stage-latency.csv` with the CPU and OpenCV version recorded.

- The end-to-end median latency and the margin against the budget.
- [out/bandwidth-model.csv](#) with the infeasible split identified.
- The protocol table: link, payload class, latency class, protocol, reason.
- The S versus S x F generative latency ratio you computed.
- The INT8 and low-rank-adaptor arithmetic.
- The privacy gate statement naming which splits it eliminates.
- [out/split-decision.md](#) including the runner-up and the deciding number.
- Three residual risks with how each would be closed.
- An explicit ILLUSTRATIVE label on the cost and energy figures.

## Troubleshooting

| Symptom                                                            | What to do                                                                                                                                                                                                                                                    |
|--------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Stage latencies vary wildly between runs                           | Possible causes include warm-up, OS load, scheduling, frequency changes or thermal limits. The harness discards the first five frames and reports a median and p95 for exactly this reason — report those, not a mean, and say how many frames were measured. |
| The generation stage cannot be timed because no model runs locally | Correct, and expected. Time the stages you actually run, and model the generation stage from the documented S and S x F relationship instead. Label it MODELLED, not MEASURED — mixing the two in one table is the mistake this lab exists to prevent.        |
| Bandwidth numbers look absurdly large                              | Check your bits-versus-bytes conversion: multiply bytes per second by 8 to get bits per second, then divide by 1e6 for Mbit/s. A factor-of-eight error here changes the architecture.                                                                         |
| Every split looks feasible                                         | Check the input dimensions, frame rate, camera count and link budget; all may legitimately fit a small fixture. Split A is defined as uncompressed frames — 1280 x 720 x 3 x 25 bytes per second per camera. That is the point of including it.               |
| The cost model produces a number you want to quote                 | Do not. It is built on indicative unit rates in a classroom constraints file. Label it ILLUSTRATIVE everywhere it appears, and state that a real figure requires current vendor pricing and a measured event rate.                                            |
| You cannot choose between two splits                               | That usually means you have not applied the privacy gate. Eliminate on constraint first, then optimise on cost among what survives.                                                                                                                           |

**Note:** The same content, plus the full prompt pack, is in [labs/lab-12-cloud-edge-architecture-tradeoffs/README.md](#) and its PDF. Sources used in this lab: S13, S15, S17, S18, S23.

## Preparing for the Assessment

**Written Assessment WA(Q&A) — 13 questions, 60 minutes, open book**

- One question per knowledge statement, K1 through K13, in that order.
- Every question is answerable from the slides. The answer key cites the slide.
- All questions are open-ended. There is no multiple choice anywhere in this assessment.
- Where a question asks you to NAME three of something, name three and say what each is for. A bare list is the minimum; the second clause is what a competent answer looks like.
- Where a question asks you to show code, show the call with its arguments — not pseudocode, and not just the function name.

### Practical Performance (PP) — 5 tasks, 90 minutes, open book

- Task 1 covers A1; Task 2 covers A2 and A3; Task 3 covers A4; Task 4 covers A5 and A6; Task 5 covers A7 and A8.
- One continuous scenario runs through all five tasks.
- Every task maps to a lab you have done. The model answer is that lab's procedure applied to the assessment scenario.
- Show your code AND your result. A result without the code that produced it cannot be assessed, and code without a result has not been run.
- Where a task asks you to explain a difference, give the mechanism, not the outcome. 'CAMShift is better' is not an answer; 'CAMShift recomputes the window size from the zeroth moment each iteration, so it adapts to a subject that changes apparent size' is.

### What to have open

- This Learner Guide.
- The slide deck PDF.
- Your own lab [out/](#) folders — your measurements are your working.
- The prompt packs in each lab's [PROMPTS.md](#).

## Glossary

- **Affine transform** — A 2x3 matrix map applied with `cv2.warpAffine`. Preserves parallel lines. Determined by three point correspondences.
- **AdaBoost** — The boosting algorithm Viola-Jones uses to select the small subset of Haar features that actually discriminate.
- **Bhattacharyya distance** — A bounded [0,1] histogram distance where 0 means identical. The brand-consistency gate in Lab 07 uses it.
- **C2PA Content Credentials** — Tamper-evident metadata attached to a media file recording origin, creator and edit history.
- **CAMShift** — Continuously Adaptive Mean Shift. Recomputes the tracking window size from the zeroth moment of the back-projection each iteration.

- **Canny** — A four-stage edge algorithm: Gaussian smoothing, Sobel gradient, non-maximum suppression, hysteresis thresholding with two thresholds.
- **CLAHE** — Contrast-limited adaptive histogram equalisation. A mapping per tile, with a clip limit that stops noise being amplified in flat regions.
- **Continuity block** — The set of prompt fields that must be character-identical across every shot of a sequence.
- **Diffusion model** — A generative model trained to reverse a fixed noise-adding process. Stable to train; slow to sample because it walks back through S steps.
- **Diffusion transformer (DiT)** — A transformer denoiser over image or video tokens; video variants model temporal relationships. Architecture alone does not guarantee coherent motion.
- **Edge density** — Non-zero Canny pixels divided by total pixels. Catches over-sharpening and over-smoothing.
- **FID / KID / LPIPS** — Distributional and perceptual metrics for generated imagery. FID is encoder-dependent; KID is unbiased at finite sample sizes; LPIPS is a pairwise perceptual distance.
- **FVD** — Frechet Video Distance — the video analogue of FID, reported with temporal consistency checks.
- **GAN** — Generative adversarial network. A generator and a discriminator in a zero-sum game. Sharp output; unstable training; prone to mode collapse.
- **HOG** — Histogram of Oriented Gradients. Cell gradient histograms, block-normalised, classified by a linear SVM. Ships in OpenCV with a trained people detector.
- **Hu moments** — Seven moment combinations theoretically invariant to translation, scale and rotation; rasterisation and segmentation introduce deviations. Guard zeros in signed-log conversion.
- **HSV** — Hue, Saturation, Value. In OpenCV 8-bit, H is 0-179 (degrees halved), S and V are 0-255. Hue may be more stable under exposure changes, but clipping, noise and lighting shifts still affect segmentation.
- **Inpainting / outpainting** — Filling a masked region inside the frame / extending the frame beyond its original boundary.
- **IoU** — Intersection over union of two boxes. Detection matching also requires the correct class, a stated threshold and one-to-one assignment.
- **Latent diffusion** — Diffusion run inside a compressed autoencoder representation rather than on pixels, which cuts memory traffic at high resolution.
- **LoRA / low-rank adapter** — Trains  $r(d+m)$  parameters instead of  $d*m$  for a  $d \times m$  layer, so a task specialisation does not duplicate the base weights.
- **Mask semantics** — Which colour of a binary mask is editable. In this course and in cv2.inpaint, white is editable and black is preserved. Always state it explicitly.
- **MQTT / CoAP** — Lightweight publish-subscribe and constrained-device protocols for sending small events from an edge node, rather than frames.
- **Mode collapse** — A GAN failure where the generator produces only a narrow set of outputs that reliably fool the discriminator.

- **Non-maximum suppression** — Greedy removal of overlapping detections by IoU, so one object yields one box.
- **ORB** — Oriented FAST plus rotated BRIEF. Designed for rotation and limited scale robustness, 32-byte binary descriptor, patent-free. Match it with Hamming distance.
- **PSNR** — Peak signal-to-noise ratio:  $10 \cdot \log_{10}(255^2 / \text{MSE})$ . A per-pixel error average that rewards smoothing.
- **Quantisation** — Reducing weight precision. Parameter memory scales by  $b/32$ , so INT8 is a quarter of FP32.
- **Reparameterisation trick** —  $z = \mu + \sigma \cdot \epsilon$ , which moves the randomness outside the gradient path so a VAE can be trained.
- **SIFT** — Scale-Invariant Feature Transform. 128-float descriptor — 512 bytes, or 16x ORB's storage per keypoint.
- **Split computing** — Dividing a pipeline between edge and cloud. The split point determines both the latency and the payload that crosses the link.
- **SSIM** — Structural similarity: luminance, contrast and structure over a sliding window. Punishes lost texture that PSNR forgives.
- **SynthID** — The watermark carried by current Gemini image outputs.
- **Temporal coherence** — Consistency ACROSS frames, as distinct from spatial coherence within one frame. The defining problem of generative video.
- **Template matching** — Sliding a patch and scoring similarity at every position. Not scale- or rotation-invariant. SQDIFF methods use minLoc; the others use maxLoc.
- **VAE** — Variational autoencoder. Encodes to a distribution rather than a point. Smooth latent space; blurry output; now the encoder-decoder inside latent diffusion.
- **Viola-Jones** — Haar features + integral image + AdaBoost + a cascade of stages. A genuine ML detector that ships inside OpenCV.

## Source Register

Every external claim in the slides and in this guide traces to one of the following. Vendor documentation was verified on 6 September 2026; model identifiers change, and the register records which were deprecated at that date. No ebook pages are reproduced — each entry records the locator and the specific fact it anchors.

| Ref | Source                                                                           | Locator                                               | What it anchors                                                                                                                                                                                                                   |
|-----|----------------------------------------------------------------------------------|-------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| S1  | Course Proposal<br>Tertiary Infotech CA-<br>WSQ-2020-013290-v2<br>(SSG-approved) | Part 4 Section E,<br>Curriculum Key<br>Features table | Approved hour<br>breakdown: 7.5 h<br>classroom facilitation +<br>6 h practical + 2.5 h<br>assessment = 16 h; per-<br>LU instructional minutes<br>(75 min CR each;<br>practical<br>0/60/60/120/60/60); WA<br>60 min and PP 90 min. |

| Ref | Source                                                                                                                                        | Locator                                                                                                                                                                                       | What it anchors                                                                                                                                                                                                                                                                                                                                               |
|-----|-----------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| S2  | Assessment Plan_OpenCV_TGS-2020505925_V1.0 (28 Dec 2020)                                                                                      | Assessment Duration table; WA and PP specification tables                                                                                                                                     | WA(Q&A) 60 min covering K1-K13; PP 90 min covering A1-A8; assessor:candidate 1:3 to 1:10; open-book; C/NYC decision rule.                                                                                                                                                                                                                                     |
| S3  | Competency Mapping.docx (TSC ICT-DIT-4022-1.1)                                                                                                | Course Mapping section and ELO/A-code matrix                                                                                                                                                  | Verbatim K1-K13 and A1-A8 statements; ELO-to-ability matrix (ELO1:A1 / ELO2:A2 / ELO3:A3,A4 / ELO4:A5 / ELO5:A6 / ELO6:A7,A8).                                                                                                                                                                                                                                |
| S4  | Computer Vision Technology.docx (Skills Framework TSC extract)                                                                                | TSC Proficiency Level 4 (ICT-DIT-4022-1.1) knowledge and abilities columns                                                                                                                    | TSC title, code, description and the authoritative knowledge/ability wording.                                                                                                                                                                                                                                                                                 |
| S5  | Tertiary Courses public registration page                                                                                                     | <a href="https://www.tertiarycourses.com.sg/wsqa-generative-ai-for-image-and-video-creation.html">https://www.tertiarycourses.com.sg/wsqa-generative-ai-for-image-and-video-creation.html</a> | Registered public title, TGS code, five delivery topics, six learning outcomes, 2-day/16-hour duration, entry requirements and target job roles.                                                                                                                                                                                                              |
| S6  | M. Tschochohei and F. Schenker, 'Multimodal Generative AI in the Enterprise: From Pixels to Profit', Packt, June 2026, ISBN 978-1-80611-167-1 | Ch.5 pp.56-62                                                                                                                                                                                 | VAE encoder/decoder and latent space; the reparameterisation trick; GAN generator/discriminator zero-sum game and mode collapse; diffusion forward (fixed Markov chain) and reverse (learned noise prediction) processes; Table 5.1 architecture comparison across image quality, sample diversity, training stability, inference speed and primary use case. |
| S7  | Tschochohei and Schenker (2026), as above                                                                                                     | Ch.5 pp.65-67                                                                                                                                                                                 | Prompt structure — subject, context, style, technical modifiers; increasing alignment with prompt length; subject lock and fixed photography style reduce output variance; structured JSON prompting for enterprise repeatability and versioning; repetition and contradictory instructions degrade output.                                                   |

| Ref | Source                                    | Locator                    | What it anchors                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|-----|-------------------------------------------|----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| S8  | Tschochohei and Schenker (2026), as above | Ch.5 pp.68-75              | Binary masking (white = editable, black = preserved); background swap; inpainting with semantic masks and mask dilation; outpainting / generative expand for aspect-ratio repurposing; mask-free editing and the DiffEdit source/target-prompt mask inference; precision-versus-speed trade-off between mask-based and mask-free edits.                                                                                                                        |
| S9  | Tschochohei and Schenker (2026), as above | Ch.5 pp.76-77 and pp.82-83 | Natively multimodal LLMs reason over text and images in one model, which is what makes character/product consistency across images tractable; enterprise deployment practices — brand style guide for AI, creative-workflow integration, review and approval gates, centralised asset management, prompt-engineering literacy.                                                                                                                                 |
| S10 | Tschochohei and Schenker (2026), as above | Ch.7 pp.106-109            | Spatial versus temporal coherence; early video-GAN failure modes (temporal artifacts/flicker, inconsistent motion, mode collapse) and VGAN/MoCoGAN/TGAN; the 2-D to 3-D U-Net extension for spatiotemporal features; Diffusion Transformers treating video as a sequence of spatiotemporal patches; the 'consistency bottleneck' argument for diffusion over GANs; platform comparison table (core technology, max duration and resolution, primary use case). |

| Ref | Source                                                                                                                                       | Locator                        | What it anchors                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|-----|----------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| S11 | Tschochohei and Schenker (2026), as above                                                                                                    | Ch.7 pp.110-119                | Text-to-video, image-to-video and video-with-sound enterprise use cases; long-running operation polling pattern; prompt-driven per-second audio specification; prompt enhancement from a campaign brief.                                                                                                                                                                                                                                                                                                   |
| S12 | Tschochohei and Schenker (2026), as above                                                                                                    | Ch.8 pp.127-132 and pp.140-143 | Modality-specific responsible-AI risks (stereotypes and representational harm, training-data and output ownership, deceptive imagery, voice cloning, deepfakes); provider safety comparison (SynthID watermarking and model cards; C2PA Content Credentials metadata; banned-word filters; configurable safety tolerance); transparency strategy — C2PA content credentials, robust invisible watermarking resilient to compression/cropping/format change, and model cards as a 'nutrition label for AI'. |
| S13 | Tschochohei and Schenker (2026), as above                                                                                                    | Ch.9 pp.150-152                | Multimodal prompt injection, steganographic payloads in images, side-channel attacks, and the Samsung data-leak example used to motivate data-protection-aware architecture choices.                                                                                                                                                                                                                                                                                                                       |
| S14 | A. Mewada, M. A. Ansari, S. Ahmad and N. Singh (eds), 'AI-Generated Image and Video Synthesis', IEEE Press / Wiley, 2026, ISBN 9781394403110 | Ch.1 pp.1-11                   | Historical evolution of synthetic media; role of data in training generative models; overview of major architectures; text-to-image, style transfer and super-resolution as distinct image-synthesis tasks; frame-based versus temporal video generation, motion transfer, and the temporal-consistency and realism challenges.                                                                                                                                                                            |

| Ref | Source                         | Locator                                                                                                                                         | What it anchors                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|-----|--------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| S15 | Mewada et al. (2026), as above | Ch.3 pp.35-44 (Vishnoi, Sisodia, Khare and Yadav, 'Sensors-to-synthesis: Edge AI and IoT for Generative Visual Systems')                        | The four-layer sensors-to-synthesis architecture (IoT sensing / edge intelligence / generative visual modelling / application) shown in Fig. 3.1; MQTT and CoAP as the lightweight sensor-to-edge protocols; split computing between edge and cloud; federated learning; Table 3.2 comparing VAE, GAN and diffusion for edge suitability; Table 3.3 domain applications; Table 3.4 challenges and future directions (compute/energy, latency, privacy/security, scalability/interoperability, ethics; pruning, quantisation, knowledge distillation, neuromorphic and FPGA accelerators, differential privacy, standardised APIs, containerised deployment). |
| S16 | Mewada et al. (2026), as above | Ch.4 pp.47-54 (Sibi, Pantola and Gupta, 'Detecting AI-generated Images in the Social Media Era: A Deep Learning Approach with GenReal Dataset') | Detection framed as a supervised binary classification problem with Grad-CAM explainability; reported test accuracies on the GenReal dataset — EfficientFormer 99.6%, FastViT and CoAt-Lite 99.1%, CoAtNet and ConvNeXt 98.7% — and on CIFAKE — CoAt-Lite Mini 98.04%, ConvNeXt 97.95%, EfficientFormer 97.75%; macro precision, recall and F1 reported alongside accuracy.                                                                                                                                                                                                                                                                                  |

| Ref | Source                            | Locator                                                                                                                              | What it anchors                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|-----|-----------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| S17 | Mewada et al. (2026),<br>as above | Ch.7 pp.94-98 (Dubey, Pinheiro, Singh, Ansari and Kumar, 'Advanced Foundations and Future Trends in Generative AI for Visual Media') | Table 7.2 scaling strategies and their targets (trajectory distillation -> fewer sampling steps at similar FID/FVD; quantisation 8/4-bit -> reduced latency and memory; low-rank adapters -> small trainable-parameter fraction; latent diffusion -> lower bandwidth cost at high resolution; retrieval conditioning -> higher faithfulness without more parameters). Evaluation protocol: FID as a 2-Wasserstein approximation in feature space (encoder-dependent, sample-size sensitive), KID as an unbiased $MMD^2$ , LPIPS as a learned perceptual distance, precision/recall for mode coverage, CLIP-based text-image faithfulness, masked-region consistency and identity preservation for edits, FVD plus optical-flow agreement for video, and watermark detectability under common edits (Table 7.3 axes and pitfalls). |

| Ref | Source                            | Locator                                                                                                                | What it anchors                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|-----|-----------------------------------|------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| S18 | Mewada et al. (2026),<br>as above | Ch.7 pp.98-100                                                                                                         | Peak memory = parameter memory + activations (+ optimiser state during training); diffusion sampling latency proportional to S denoising steps for images and to S x F for video; moving weights from 32-bit to b-bit scales parameter memory by b/32; low-rank adapters train $r(d+m)$ parameters versus $d*m$ ; energy per sample approximated by overhead factor x energy-per-FLOP x FLOPs-per-sample, with hardware, kWh, PUE, grid carbon intensity and joules-per-sample all reportable; Table 7.4 deployment decision matrix (low-latency image -> distilled sampler + INT8; edge device -> INT8 + PEFT adapters; high resolution -> latent diffusion + few-step sampler; short video -> temporal attention + distilled steps; brand/style control -> retrieval conditioning + filters; tight privacy -> local open model + provenance tracking). |
| S19 | Mewada et al. (2026),<br>as above | Ch.11 pp.147-160<br>(Dewang, Mewada, Omkar, Jaiswal and Singh, 'Hybrid Neural Networks for Robust Deepfake Detection') | Reported validation accuracies for deepfake detection backbones — Xception 77.5%, DenseNet121 and ACNN-RAN 75.5%, VGG19 73.5% — with precision, recall, F1 and AUC reported together, illustrating that headline accuracy alone hides class-wise behaviour on a hard, shifted distribution.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |

| Ref | Source                                                               | Locator                                                                                                                                           | What it anchors                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|-----|----------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| S20 | Google AI for Developers — Gemini API image generation documentation | <a href="https://ai.google.dev/gemini-api/docs/image-generation">https://ai.google.dev/gemini-api/docs/image-generation</a> (verified 6 Sep 2026) | Current image-generation model IDs gemini-3.1-flash-image, gemini-3.1-flash-lite-image, gemini-3-pro-image, with gemini-2.5-flash-image as the legacy model; the client.interactions.create call pattern for text-to-image, image editing and multi-turn conversational editing via previous_interaction_id; aspect ratios including 1:1, 3:2, 2:3, 3:4, 4:3, 4:5, 5:4, 9:16, 16:9 and 21:9; output sizes 1K/2K/4K by model; and the statement that all generated images include a SynthID watermark. |
| S21 | Google AI for Developers — Gemini API Imagen documentation           | <a href="https://ai.google.dev/gemini-api/docs/imagen">https://ai.google.dev/gemini-api/docs/imagen</a> (verified 6 Sep 2026)                     | CURRENCY CORRECTION: the imagen-4.0-generate-001 family used in the reference ebook is documented as deprecated with shutdown on 17 August 2026, and the Gemini native image models are the documented replacement. Course materials therefore teach the current Gemini image model IDs and treat the ebook's Imagen code as historical.                                                                                                                                                              |

| Ref | Source                                                        | Locator                                                                                                                 | What it anchors                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|-----|---------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| S22 | Google AI for Developers — Veo video generation documentation | <a href="https://ai.google.dev/gemini-api/docs/veo">https://ai.google.dev/gemini-api/docs/veo</a> (verified 6 Sep 2026) | Current Veo model IDs veo-3.1-generate-preview, veo-3.1-fast-generate-preview and veo-3.1-lite-generate-preview, with veo-3.0-generate-001 deprecated; native audio generation; durationSeconds values 4, 6 and 8; aspectRatio 16:9 (default) and 9:16; resolution 720p (default), 1080p and 4k with Lite limited to 720p/1080p; up to three reference images to guide content on supported non-Lite models; image-to-video across versions. 1080p/4k require 8-second duration; extension is 720p and adds 7 seconds. Lite does not support referenceImages or video extension. The client.models.generate_videos long-running-operation polling pattern. |

| Ref | Source                                                             | Locator                              | What it anchors                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|-----|--------------------------------------------------------------------|--------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| S23 | OpenCV 4.13 Python package, verified locally on the delivery image | cv2 4.13.0 (opencv-python 4.13.0.92) | API-currency verification for every command taught: cv2.data.harcascades ships the frontal-face and eye cascades so no weights download is required; cv2.HOGDescriptor_getDefaultPeopleDetector() is built in; SIFT_create, ORB_create, FastFeatureDetector_create, cornerHarris, goodFeaturesToTrack, matchTemplate, createBackgroundSubtractorMOG2/KNN, meanShift, CamShift, calcOpticalFlowFarneback and PSNR are all present. IMPORTANT CORRECTION carried into the labs: cv2.TrackerCSRT_create and cv2.TrackerKCF_create are NOT in the base opencv-python wheel (they require opencv-contrib-python), and the cv2.quality SSIM module is also contrib-only — so the labs use cv2.TrackerMIL_create and a NumPy SSIM implementation to stay dependency-light and runnable offline. |